

Redes Neuronales LSTM para el análisis de parámetros econofísicos en series de tiempo financieras

Robinson Manuel Avila Hernandez

March 4, 2025

Abstract

[1] En este trabajo se desarrolla un modelo basado en redes neuronales inspirándose por [2] pero realizando el código en python y no en R, usando redes neuronales que usan el modelo LSTM (Long Short-Term Memory), la cual es muy utilizada en el mundo financiero para predecir históricos de precio como el S&P500, lo cual fue lo primero en corroborarse en este proyecto, utilizando los datos del histórico de precios del S&P500 de los últimos 34 años (1 de enero de 1930 hasta 1 de enero de 2024) se logro predecir con precisión el comportamiento del histórico de precios, usando una red neuronal de tipo LSTM, mostrando así que se puede crear una red neuronal tipo LSTM, con los recursos disponibles, para después proceder con los objetivos de este proyecto, los cuales son usar redes neuronales tipo LSTM, para predecir la serie temporal de parámetros mas complejos de la econofísica, en series de precio como la del S&P500, esos parámetros son: el índice de Hurts, la entropía de Shannon, la fractalidad y la volatilidad, lo cual se logro satisfactoriamente con las herramientas de cómputo otorgadas por Google colab.

1 Introducción

El análisis de mercados financieros ha sido un tema de gran interés, especialmente en la predicción de precios mediante técnicas computacionales. En este proyecto, proponemos

el uso de redes neuronales para la predicción de parámetros econofísicos como la entropía de Shannon, el índice de Hurst, la fractalidad y la volatilidad, haciendo uso de redes neuronales de tipo LSTM las cuales son de uso cotidiano en el análisis financiero como para la predicción sobre comportamiento del histórico de precios de activos financieros como por ejemplo el S&P500.

[2] Hagamos un repaso histórico sobre como las redes neuronales se empezaron a usar para la predicción del comportamiento económico: La historia del uso de redes neuronales en el análisis predictivo de mercados financieros, como el S&P 500, se remonta a las bases teóricas de los años 40 y 50 con el modelo de neurona artificial de McCulloch y Pitts y el perceptrón de Rosenblatt, aunque su aplicación práctica en economía comenzó en los 90, cuando el avance computacional permitió explorar redes neuronales multicapa y backpropagation para capturar relaciones no lineales en series temporales financieras, como demuestran estudios pioneros de los años 90 que comparaban su eficacia frente a modelos estadísticos tradicionales. En los 2000, bancos y fondos de inversión como Renaissance Technologies integraron redes neuronales con métodos clásicos de machine learning, aunque su adopción masiva llegó en la década de 2010 con el deep learning, GPUs y grandes conjuntos de datos, permitiendo aplicar arquitecturas complejas como LSTM para secuencias temporales, CNN para patrones gráficos y transformers para análisis de largo plazo, combinando datos históricos de precios,

volumen, noticias y sentimiento de redes sociales. A pesar de su potencial, su uso enfrenta desafíos como la hipótesis de mercado eficiente, el sobre ajuste a ruido financiero y la falta de interpretabilidad, lo que ha llevado a soluciones como validación cruzada temporal y modelos híbridos. Hoy, su aplicación incluye desde high-frequency trading hasta la simulación de crisis con GANs, mientras el futuro apunta a técnicas como reinforcement learning para estrategias adaptativas y quantum machine learning para modelar complejidades extremas, consolidando a las redes neuronales como herramientas indispensables, aunque controvertidas, en la predicción financiera moderna.

Ahora veamos un poco como se involucra la física en el mundo de la economía y especialmente en el mundo del uso de las redes neuronales para la predicción en la economía. La física se integra en la economía a través de disciplinas como la econofísica, que aplica principios y métodos físicos —como la mecánica estadística, los sistemas complejos y los procesos estocásticos— para modelar comportamientos económicos y financieros, especialmente en mercados caóticos y dinámicos. En el ámbito de las redes neuronales, herramientas propias de la física, como las ecuaciones diferenciales estocásticas, los modelos de Ising (usados para simular interacciones entre agentes) o el análisis de fluctuaciones críticas, enriquecen el diseño de algoritmos predictivos al capturar no linealidades, patrones temporales y correlaciones ocultas en datos económicos. Las redes neuronales, inspiradas en sistemas biológicos pero matematizadas con enfoques físicos, aprovechan conceptos como la optimización de energía (análoga a la minimización de funciones de pérdida) y la termodinámica de la información, haciendo uso de parámetros como la entropía de Shannon, la fractalidad, etc; para procesar grandes volúmenes de datos, prever tendencias macroeconómicas, gestionar riesgos financieros o simular escenarios bajo incertidumbre. Además, técnicas como el machine learning híbrido, que combina redes

neuronales con modelos físicos, permiten mejorar la interpretabilidad y robustez de las predicciones, adaptándose a la volatilidad de los mercados con un enfoque interdisciplinario que equilibra teoría económica, dinámica de sistemas y ciencia de datos.

[1] Las LSTM son un tipo de **Red Neuronal Recurrente (RNN)** diseñada para resolver el problema del **desvanecimiento del gradiente**.

Componentes Clave

- **Celda de memoria:** Almacena información a largo plazo (C_t)
- **Puertas:** Regulan flujo de información mediante sigmoide (σ)

Ecuaciones Fundamentales

Puerta de Olvido: $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$

Puerta de Entrada: $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$

Candidato de memoria: $\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$

Puerta de Salida: $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$

Actualización de celda: $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$

Estado oculto: $h_t = o_t \odot \tanh(C_t)$

[3] La entropía de Shannon para una variable aleatoria discreta con distribución de probabilidad $P(x_i)$ está definida como:

$$S = - \sum_i P(x_i) \log P(x_i) \quad (1)$$

donde $P(x_i)$ es la probabilidad del estado x_i . En el contexto de series de tiempo financieras, x_i representa los retornos de los precios en una ventana de tiempo.

[5] La fractalidad, el exponente de Hurst (los cuales serán definidos mas adelante en su apartado teórico) tienen una estrecha relación sobre como medir la complejidad de predicción y por lo tanto la correlación temporal de una serie de tiempo, mostrándolo a través de los siguientes

valores (ver apartado teórico mas adelante para entender un poco mas a profundidad):

- Eficiencia de mercado ($H \approx 0.5$)
- Persistencia ($H > 0.5$): Mercados con tendencias
- Anti-persistencia ($H < 0.5$): Mercados mean-reverting

Este trabajo explora la aplicación de la geometría fractal al análisis de mercados financieros. Mediante el cálculo de la dimensión fractal en series históricas de precios, demostramos cómo esta métrica cuantifica la complejidad y predictibilidad de mercados, ofreciendo insights sobre eficiencia de mercado, gestión de riesgos y detección de crisis. Se presenta un marco teórico-matemático junto con aplicaciones prácticas usando datos reales.

2 Marco Teórico

2.1 Teoría del modelo LSTM de red neuronal (Long Short-Term Memory)

[1] Las LSTM son un tipo de **Red Neuronal Recurrente (RNN)** diseñada para resolver el problema del **desvanecimiento del gradiente**. Componentes Clave

- **Celda de memoria:** Almacena información a largo plazo (C_t)
- **Puertas:** Regulan flujo de información mediante sigmoide (σ)

Ecuaciones Fundamentales de LSTM

2.1.1 Puerta de Olvido (Forget Gate)

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (2)$$

Esta puerta decide qué información de C_{t-1} se descarta.

2.1.2 Puerta de Entrada (Input Gate)

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (3)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (4)$$

Aquí, i_t indica la importancia de nueva información y $\tilde{C}_t$ es el candidato de actualización.

2.1.3 Actualización de la Celda de Memoria

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (5)$$

Donde $\odot$ representa la multiplicación elemento a elemento.

2.1.4 Puerta de Salida (Output Gate)

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (6)$$

$$h_t = o_t \odot \tanh(C_t) \quad (7)$$

Esta puerta controla qué parte de la celda de memoria se envía como salida.

Ahora se verán las puertas en conjunto y se explicara cada uno de sus componentes matemáticos.

$$\begin{aligned} f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f), \\ i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i), \\ \tilde{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c), \\ c_t &= f_t c_{t-1} + i_t \tilde{c}_t, \\ o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o), \\ h_t &= o_t \tanh(c_t), \end{aligned}$$

donde: $x_t \in R^d$ es la entrada en el paso t . $h_{t-1} \in R^h$ es la salida (oculta) en el paso anterior. $c_t \in R^h$ es el estado interno (memoria) en el paso t . $\sigma(\cdot)$ es la función sigmoide (logística), $\sigma(z) = \frac{1}{1+e^{-z}}$. $\tanh(\cdot)$ es la función tangente hiperbólica. W_f, W_i, W_c, W_o y b_f, b_i, b_c, b_o son matrices y vectores de parámetros aprendibles.

Rol de Cada Puerta:

1. Puerta de Olvido (f_t): Controla cuánto del estado c_{t-1} se retiene. 2. Puerta de Entrada (i_t): Controla cuánta información nueva ingresa en la

memoria. 3. Contenido Candidato ($\tilde{c}_t$): Representa la nueva información potencial a añadir. 4. Puerta de Salida (o_t): Decide cuánto de la memoria se expone como salida.

2. Derivación y Significado de las Puertas Lógicas

2.1. Puerta de Olvido (f_t)

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f).$$

- En redes RNN tradicionales, el estado se actualiza sin un mecanismo que decida qué información descartar. - Para evitar que la memoria se llene de datos irrelevantes, se introduce un factor multiplicativo f_t . - La función $\sigma(\cdot)$ produce valores en $[0, 1]$, perfectos para escalar la memoria.

2.2. Puerta de Entrada (i_t)

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i).$$

- Indica qué fracción de la nueva información entra a la memoria. - Multiplica $\tilde{c}_t$ para añadir la parte relevante de la nueva información.

2.3. Contenido Candidato ($\tilde{c}_t$)

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c).$$

- Produce valores en $[-1, 1]$. - Representa la información candidata que podría entrar a la memoria.

2.4. Estado de Memoria (c_t)

$$c_t = f_t c_{t-1} + i_t \tilde{c}_t.$$

- Permite que la LSTM mantenga valores importantes (cuando $f_t \approx 1$) y evitar el desvanecimiento del gradiente.

2.5. Puerta de Salida (o_t) y Salida (h_t)

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad h_t = o_t \tanh(c_t).$$

- Controla cuánto de la memoria se “expone” al exterior.

3. Desarrollo Matemático de la Retro-propagación

3.1. Gradiente de c_t respecto a c_{t-1} :

$$\frac{\partial c_t}{\partial c_{t-1}} = \frac{\partial}{\partial c_{t-1}}(f_t c_{t-1} + i_t \tilde{c}_t) = f_t.$$

Si $f_t \approx 1$, la memoria se transfiere sin decaer.

3.2. Gradiente de h_t con respecto a c_t :

$$\frac{\partial h_t}{\partial c_t} = o_t \cdot \text{sech}^2(c_t) = o_t(1 - \tanh^2(c_t)).$$

El producto de gradientes en el tiempo incluye f_t , mitigando el vanishing gradient.

4. Importancia en Econofísica

1. **Dependencias de Largo Plazo:** Las crisis financieras pueden desarrollarse en horizontes prolongados y la LSTM retiene esas “memorias”. 2. **Procesos Estocásticos:** Al permitir una retención controlada de la información, la LSTM puede modelar fluctuaciones y regímenes en series económicas. 3. **Evitar Vanishing Gradient:** Vital para entrenar redes profundas en contextos financieros.

Para una explicación mas didáctica ver [10], ahora podemos ver una representación gráfica en la figura 2, de las tres puertas lógicas de una celda, de la cadena de celdas que componen la red neuronal de tipo LSTM.

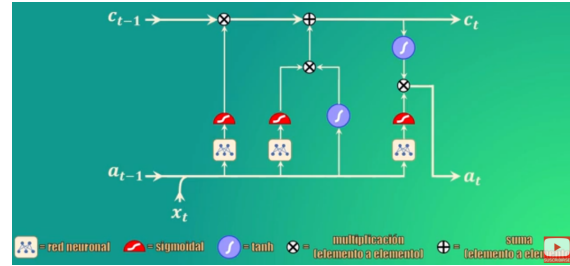


Figure 1: Diagrama de las tres puertas lógicas de una celda de red neuronal tipo LSTM, imagen obtenida de [10].

2.1.5 ¿Por qué Funcionan?

• Gradientes Estables:

$$\frac{\partial C_t}{\partial C_{t-1}} = f_t \quad (\text{Evita desvanecimiento})$$

• No linealidades:

$$\sigma(x) \in (0, 1), \quad \tanh(x) \in (-1, 1)$$

- **Interacción Dinámica:**

$$\begin{cases} f_t \approx 0 & \Rightarrow \text{Reset celda} \\ i_t, f_t \text{ balanceados} & \Rightarrow \text{Memoria a largo plazo} \end{cases}$$

2.1.6 Ejemplo Numérico

Datos iniciales:

$$C_{t-1} = 2.5, h_{t-1} = 0.8, x_t = 1.0$$

Cálculos:

$$C_t = 0.9 \times 2.5 + 0.5 \times 0.6 = 2.55$$

$$h_t = 0.7 \times \tanh(2.55) \approx 0.686$$

2.1.7 Resumen

- **Sistema de Memoria:**

$$\text{Puertas} \rightarrow \text{Celda} \rightarrow h_t$$

- **Clave Matemática:**

$$\begin{cases} \text{Control:} & \sigma(W \cdot [h_{t-1}, x_t] + b) \\ \text{Memoria:} & \tanh(\cdot) \text{ y operaciones } \odot \end{cases}$$

2.1.8 Ventajas sobre RNN estándar

1. Manejo eficiente de gradientes mediante puertas
2. Control selectivo de información relevante
3. Capacidad para aprender dependencias a largo plazo

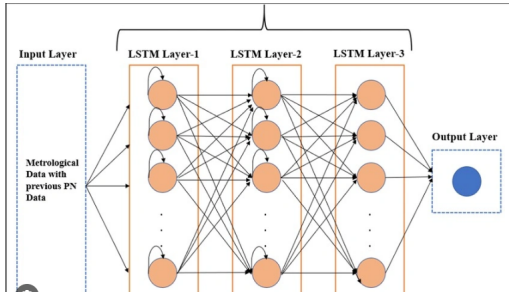


Figure 2: Diagrama de flujo del modelo LSTM.

2.1.9 Aplicaciones

- Procesamiento de lenguaje natural (NLP)
- Reconocimiento de voz
- Predicción de series temporales (Aplicación usada en este caso)

2.2 Teoría de la entropía de Shanon en Econofísica

[3] La entropía de Shannon para una variable aleatoria discreta con distribución de probabilidad $P(x_i)$ está definida como:

$$S = - \sum_i P(x_i) \log P(x_i) \quad (8)$$

donde $P(x_i)$ es la probabilidad del estado x_i . En el contexto de series de tiempo financieras, x_i representa los retornos de los precios en una ventana de tiempo.

2.2.1 Interpretación en Econofísica

En econofísica, la entropía de Shannon se aplica al análisis de series de tiempo financieras para medir la complejidad y eficiencia de los mercados. Se utiliza para:

- Evaluar el grado de aleatoriedad en los retornos financieros.
- Analizar la eficiencia del mercado en términos de informador.
- Identificar patrones de comportamiento en la evolución de precios.

Si un mercado es completamente eficiente, los retornos financieros siguen un proceso de caminata aleatoria y su entropía es alta. En contraste, si se identifican patrones predecibles en los retornos, la entropía disminuye, indicando posibles oportunidades de arbitraje.

2.2.2 Cálculo en Series de Tiempo Financieras

Para una serie de tiempo de precios P_t , se calculan los retornos logarítmicos:

$$r_t = \log \frac{P_t}{P_{t-1}} \quad (9)$$

A partir de los valores de r_t , se construye un histograma de frecuencias normalizado que proporciona una estimación de la distribución de probabilidad $P(r_i)$. La entropía de Shannon se calcula como:

$$S = - \sum_i P(r_i) \log P(r_i) \quad (10)$$

donde $P(r_i)$ es la probabilidad de que el retorno se encuentre en el bin i del histograma.

2.2.3 Interpretación de Resultados

Los valores de entropía pueden clasificarse en tres niveles:

- **Alta entropía:** El mercado es altamente eficiente y los precios siguen una caminata aleatoria.
- **Baja entropía:** Existen estructuras predictivas en los datos, lo que puede indicar ineficiencias en el mercado.
- **Entropía intermedia:** Un balance entre aleatoriedad y estructura, común en mercados en transición.

2.3 Teoría de la Fractalidad en Econofísica: Análisis de Dimensiones Fractales en Series Temporales Financieras

[5] La Econofísica utiliza herramientas de física estadística para modelar sistemas económicos complejos. Los mercados financieros exhiben propiedades fractales donde:

- **Autosimilitud:** Patrones se repiten en distintas escalas temporales

- **Rugosidad:** Irregularidades persistentes en series de precios

- **Dimensión Fractal (D):** Cuantifica complejidad de la serie ($1 < D < 2$)

Base Matemática:

2.3.1 Dimensión Fractal (Mandelbrot, 1967)

Para una serie temporal de precios $P(t)$, la dimensión fractal se calcula como:

$$D = \lim_{\epsilon \rightarrow 0} \frac{\log N(\epsilon)}{\log(1/\epsilon)}$$

Donde $N(\epsilon)$ es el número de cajas de tamaño ϵ necesarias para cubrir la serie.

2.3.2 Relación con el Exponente de Hurst (H)

[4] Para series financieras:

$$D = 2 - H$$

2.3.3 Derivación de la Relación $D = 2 - H$

La relación entre la dimensión fractal D y el exponente de Hurst H se origina en el estudio del movimiento browniano fraccional y la teoría de series temporales.

2.3.4 1. Definición del Exponente de Hurst

El exponente de Hurst H mide la persistencia o anti-persistencia de una serie de tiempo y se define a partir del análisis de la relación de escalamiento entre la varianza R/S de la serie y el número de observaciones n :

$$E \left[\frac{R(n)}{S(n)} \right] \propto n^H$$

donde $R(n)$ es el rango acumulado de la serie y $S(n)$ su desviación estándar.

2.3.5 2. Dimensión Fractal y Autosemejanza

La dimensión fractal D caracteriza la irregularidad de una trayectoria en un espacio métrico. Para un proceso autosemejante con dependencia temporal, la dimensión fractal está relacionada con la ley de escala de la longitud de trayectoria $L(\epsilon)$, medida a una resolución ϵ :

$$L(\epsilon) \propto \epsilon^{1-D}$$

Para un proceso estocástico, como el movimiento browniano fraccional, la trayectoria satisface la relación de escalamiento:

$$L(n) \propto n^H$$

2.3.6 3. Relación entre D y H

Dado que la longitud de la trayectoria obedece la relación de escala:

$$L(\epsilon) \propto \epsilon^{1-D}$$

y sabiendo que para un proceso autosemejante $L(n) \propto n^H$, igualando exponentes obtenemos:

$$1 - D = H$$

Por lo tanto, despejando D :

$$D = 2 - H$$

2.3.7 4. Interpretación Física

- Para un **movimiento browniano estándar**, $H = 0.5$ y $D = 1.5$, indicando una trayectoria irregular. - Para un **proceso más persistente** ($H > 0.5$), la dimensión fractal disminuye, indicando una trayectoria más suave. - Para un **proceso anti-persistente** ($H < 0.5$), la dimensión fractal aumenta, indicando mayor rugosidad.

Esta relación es clave en el análisis de series temporales financieras, donde H permite caracterizar mercados eficientes o con memoria de largo plazo.

- $H = 0.5$: Mercado eficiente (paseo aleatorio)
- $H > 0.5$: Serie persistente (tendencias)
- $H < 0.5$: Serie anti-persistente (reversiones)

2.3.8 Implicaciones Financieras

2.3.9 Interpretación de la Dimensión Fractal

$$\begin{cases} D \approx 1.5 & \text{Mercado eficiente (Browniano)} \\ D > 1.5 & \text{Mayor rugosidad, menor predictibilidad} \\ D < 1.5 & \text{Estructuras persistentes, posibles arbitrajes} \end{cases}$$

2.3.10 Escalamiento Multifractal

En un proceso monofractal, la estructura de escalamiento de los momentos de orden q sigue una relación lineal con un único exponente de Hurst H . Sin embargo, en un proceso multifractal, la relación de escalamiento varía dependiendo de la región de la serie analizada.

La estructura general para los momentos de orden q está dada por:

$$\langle |X(t + \Delta t) - X(t)|^q \rangle \sim \Delta t^{\tau(q)} \quad (11)$$

donde $\tau(q)$ es la función de escalamiento, que describe cómo los momentos de la serie se comportan en diferentes escalas de tiempo.

2.3.11 Relación con el Exponente de Hurst

Para un proceso monofractal, la función de escalamiento $\tau(q)$ está relacionada con el exponente de Hurst H mediante:

$$\tau(q) = qH - D(q) \quad (12)$$

donde $D(q)$ es una función que caracteriza la multifractalidad del proceso. En el caso de un proceso estrictamente monofractal, $D(q)$ es constante, y la relación se reduce a:

$$\tau(q) = qH \quad (13)$$

Sin embargo, en procesos multifractales, $D(q)$ no es constante, lo que introduce una dependencia no lineal en la función $\tau(q)$.

2.3.12 Interpretación de la Multifractalidad

- Si $\tau(q)$ es una función lineal en q , el proceso es monofractal y está completamente caracterizado por un único exponente de Hurst H . - Si $\tau(q)$ es no lineal, el proceso es multifractal, lo que indica que existen diferentes escalas de irregularidad en la serie temporal. - La multifractalidad se manifiesta en mercados financieros, donde la volatilidad y las fluctuaciones de precios presentan diferentes grados de dependencia temporal en distintas escalas.

La función $\tau(q)$ está relacionada con el espectro de singularidades $f(\alpha)$ mediante la transformada de Legendre:

$$\alpha = \frac{d\tau(q)}{dq}, \quad f(\alpha) = q\alpha - \tau(q) \quad (14)$$

Este análisis permite caracterizar la complejidad de series temporales económicas y financieras, proporcionando una forma de cuantificar la irregularidad en múltiples escalas.

2.3.13 Aplicación a Series Temporales Financieras

Las series temporales financieras presentan características estadísticas complejas, como memoria de largo plazo y multifractalidad, lo que las hace incompatibles con modelos clásicos como el movimiento browniano estándar. En su lugar, se utiliza el movimiento browniano fraccional (fBm), que generaliza el movimiento browniano estándar al incorporar dependencias de largo alcance.

2.3.14 Definición del Movimiento Browniano Fraccional

El movimiento browniano fraccional $B_H(t)$ es un proceso gaussiano con media cero y covarianza definida por:

$$E[B_H(t)B_H(s)] = \frac{1}{2} (|t|^{2H} + |s|^{2H} - |t-s|^{2H}) \quad (15)$$

donde H es el exponente de Hurst, que controla la persistencia del proceso:

- Si $H = 0.5$, se recupera el movimiento browniano estándar (sin memoria). - Si $H > 0.5$, el proceso tiene memoria persistente (las tendencias tienden a continuar). - Si $H < 0.5$, el proceso es anti-persistente (las tendencias tienden a revertirse).

2.3.15 Derivación de la Representación Integral del Movimiento Browniano Fraccional

El movimiento browniano fraccional (fBm) $B_H(t)$ es un proceso estocástico que generaliza el movimiento browniano estándar introduciendo memoria y dependencia a largo plazo. Puede ser definido en términos de una convolución de un proceso de Wiener $B(s)$ con un núcleo de memoria.

2.3.16 Definición del Movimiento Browniano Fraccional

El fBm es un proceso gaussiano con media cero y función de covarianza dada por:

$$E[B_H(t)B_H(s)] = \frac{1}{2} (|t|^{2H} + |s|^{2H} - |t-s|^{2H}), \quad (16)$$

donde $H \in (0,1)$ es el índice de Hurst, que determina la rugosidad y la persistencia del proceso.

2.3.17 Construcción del fBm como Integral de Wiener

La representación integral del fBm se obtiene al expresar $B_H(t)$ en términos del proceso de Wiener estándar $B(s)$, utilizando la convolución con un núcleo de memoria $K_H(t,s)$.

$$B_H(t) = \int_{-\infty}^t K_H(t, s) dB(s). \quad (17)$$

El núcleo $K_H(t, s)$ se obtiene imponiendo que $B_H(t)$ tenga la covarianza deseada. Se puede derivar a partir de la teoría de integrales fraccionarios y procesos gaussianos.

2.3.18 Forma Explícita del Núcleo de Memoria

El núcleo de memoria adecuado para el movimiento browniano fraccional es:

$$K_H(t, s) = \frac{1}{\Gamma(H + \frac{1}{2})} (t - s)^{H - \frac{1}{2}} 1_{s < t}, \quad (18)$$

donde $1_{s < t}$ es la función indicadora que asegura que la integral solo se tome para valores pasados de s . Este núcleo proviene de la representación de Riemann-Liouville de los procesos fraccionarios.

2.3.19 Representación Integral Final

Sustituyendo $K_H(t, s)$ en la ecuación integral, obtenemos la representación final:

$$B_H(t) = \frac{1}{\Gamma(H + \frac{1}{2})} \int_{-\infty}^t (t - s)^{H - \frac{1}{2}} dB(s). \quad (19)$$

Esta expresión describe cómo el movimiento browniano fraccional puede construirse a partir del proceso de Wiener con memoria, lo que permite analizar su estructura fractal y sus propiedades de dependencia a largo plazo.

donde:

- $\Gamma(x)$ es la función gamma, que generaliza el factorial a números reales. - $(t - s)^{H - \frac{1}{2}}$ es un núcleo de memoria que pondera las contribuciones del pasado. - $dB(s)$ representa el diferencial del proceso de Wiener estándar.

2.3.20 Interpretación de la Representación Integral

Esta representación muestra que el fBm no es una suma de incrementos independientes (como el movimiento browniano estándar), sino que cada valor presente depende de toda la historia pasada, con un peso determinado por el exponente de Hurst H .

- Para $H = 0.5$, el peso $(t - s)^{H - \frac{1}{2}}$ es constante, lo que implica ausencia de memoria. - Para $H > 0.5$, los valores pasados tienen una influencia más fuerte en los valores futuros, lo que introduce persistencia. - Para $H < 0.5$, los valores pasados tienden a invertir la dirección del movimiento futuro, creando anti-persistencia.

2.3.21 Relevancia en Finanzas

El fBm es ampliamente utilizado en modelos financieros debido a su capacidad para capturar efectos de memoria en los retornos de activos:

- Permite modelar la persistencia observada en tendencias de mercado. - Explica la aparición de volatilidad agrupada en series financieras. - Justifica desviaciones de la hipótesis del paseo aleatorio, proporcionando una mejor descripción de mercados ineficientes.

Su aplicación ha sido clave en el desarrollo de modelos de precios de activos que extienden el modelo de Black-Scholes al contexto de mercados con memoria.

2.3.22 Aplicaciones Prácticas

1. Detección de Regímenes de Mercado
2. Optimización de Horizonte Temporal en Trading
3. Cálculo de Riesgo Adaptativo (Value-at-Risk Fractal)
4. Identificación de Burbujas Especulativas

2.4 Teoría del Exponente de Hurst y la Correlación Temporal

El exponente de Hurst (H) es un parámetro fundamental en el análisis de series temporales, utilizado para caracterizar la memoria a largo plazo y la persistencia en los datos. Su origen proviene del estudio de los procesos estocásticos fraccionarios y tiene aplicaciones en física estadística, finanzas y geofísica.

2.4.1 Definición Matemática del Exponente de Hurst

El exponente de Hurst se define a partir del análisis de rango reescalado (R/S), el cual sigue la relación de escalamiento:

$$R/S(n) = C \cdot n^H, \quad (20)$$

donde:

- $R(n)$ es el rango de la serie acumulativa dentro de una ventana de tamaño n .
- $S(n)$ es la desviación estándar dentro de la misma ventana.
- C es una constante del sistema.
- H es el exponente de Hurst, que caracteriza la memoria de la serie.

El valor de H tiene una interpretación clave en términos de dependencia temporal:

- $0 < H < 0.5$: La serie es anti-persistente, lo que significa que una tendencia creciente es probablemente seguida por una tendencia decreciente y viceversa.
- $H = 0.5$: La serie sigue un proceso de ruido blanco o movimiento browniano estándar, sin memoria temporal.
- $0.5 < H < 1$: La serie es persistente, lo que significa que una tendencia creciente tiende a continuar en el mismo sentido.

2.4.2 Derivación del Método R/S

El método clásico del análisis R/S consiste en los siguientes pasos:

1. Consideremos una serie temporal X_t con $t = 1, \dots, N$.
2. Dividimos la serie en ventanas de tamaño n .
3. Para cada ventana de tamaño n , calculamos la media $\bar{X}_n$:

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i. \quad (21)$$

4. Construimos la serie acumulativa centrada en la media:

$$Y_t = \sum_{i=1}^t (X_i - \bar{X}_n). \quad (22)$$

5. Definimos el rango dentro de la ventana n como:

$$R(n) = \max(Y_1, Y_2, \dots, Y_n) - \min(Y_1, Y_2, \dots, Y_n). \quad (23)$$

6. Calculamos la desviación estándar $S(n)$:

$$S(n) = \sqrt{\frac{1}{n} \sum_{i=1}^n (X_i - \bar{X}_n)^2}. \quad (24)$$

7. El valor de $R/S(n)$ se obtiene dividiendo el rango por la desviación estándar:

$$R/S(n) = \frac{R(n)}{S(n)}. \quad (25)$$

8. La relación entre $R/S(n)$ y n sigue una ley de potencias dada por:

$$R/S(n) \sim C \cdot n^H. \quad (26)$$

9. Aplicando logaritmos en ambos lados obtenemos:

$$\log(R/S(n)) = H \log(n) + \log(C). \quad (27)$$

10. El exponente de Hurst se estima a partir de la pendiente del ajuste lineal en un gráfico $\log(R/S)$ contra $\log(n)$:

$$H = \frac{\log(R/S)}{\log(n)} + \epsilon. \quad (28)$$

2.4.3 Importancia del Uso de Ventanas en la Estimación de H

Dado que los procesos financieros como el S&P 500 son dinámicos y pueden exhibir cambios estructurales en el tiempo (regímenes de volatilidad, crisis económicas, tendencias alcistas o bajistas), la estimación del exponente de Hurst no puede considerarse estacionaria a lo largo de toda la serie. Para capturar correctamente la variabilidad de H , se introduce la técnica de **ventanas deslizantes**.

Definición de la Ventana Se define una ventana de longitud N_w , dentro de la cual se calcula H . Luego, la ventana se desliza a lo largo de la serie temporal en incrementos de ΔN_w , obteniendo así una serie de valores de H en función del tiempo:

$$H_t = H(X_t, X_{t+1}, \dots, X_{t+N_w}). \quad (29)$$

Justificación Matemática del Uso de Ventanas

1. ****Reducción de la variabilidad local****: Al dividir la serie en ventanas de tamaño N_w , se minimizan los efectos de fluctuaciones espurias en escalas pequeñas, proporcionando una estimación más robusta de H .

2. ****Captura de la dinámica temporal****: Los mercados financieros no son estacionarios. Si calculáramos H sobre toda la serie, asumiríamos que la dependencia temporal es constante en todo el período analizado, lo cual es incorrecto.

Usar ventanas permite detectar cambios en la persistencia de la serie.

3. ****Optimización del balance entre sesgo y varianza****: - Si N_w es demasiado grande, se obtiene una estimación muy estable de H pero se pierde la capacidad de capturar variaciones locales. - Si N_w es demasiado pequeño, la estimación se vuelve ruidosa y poco confiable. - Un compromiso adecuado se elige típicamente en función del número total de datos y la naturaleza de la serie. En estudios financieros, se usan comúnmente entre 50 y 200 ventanas.

4. ****Conexión con la Transformada de Wavelet y DFA****: La idea de descomponer una serie en segmentos está relacionada con la Transformada de Wavelet y el método de Análisis de Fluctuaciones Detrendizadas (DFA), donde se estudia cómo la variabilidad en diferentes escalas afecta la estimación de H .

2.4.4 Método R/S con Ventanas Deslizantes

Dado un conjunto de datos $X(t)$, se sigue el siguiente procedimiento:

1. Dividimos la serie en segmentos de longitud N_w .
2. Para cada segmento, calculamos $R(n)$, $S(n)$, y su relación $R/S(n)$.
3. Ajustamos el modelo $R/S(n) = C \cdot n^H$ en cada ventana.
4. Obtenemos H_t para cada ventana y graficamos su evolución en el tiempo.

La variación de H_t nos permite estudiar cómo cambia la persistencia en los mercados financieros en distintos períodos, identificando posibles anomalías o regímenes de volatilidad.

2.4.5 Método de Diferenciación para el Exponente de Hurst

Otra forma de estimar H es a partir del crecimiento de la varianza en diferentes escalas de

tiempo. Para una serie temporal $X(t)$, la varianza de los incrementos sigue la ley:

$$\text{Var}(X(t + \Delta t) - X(t)) \sim (\Delta t)^{2H}. \quad (30)$$

Tomando logaritmos en ambos lados,

$$\log(\text{Var}(X(t + \Delta t) - X(t))) = 2H \log(\Delta t) + \text{constante} \quad (31)$$

Diferenciando con respecto a $\log(\Delta t)$:

$$H = \frac{1}{2} \frac{d \log(\text{Var}(X(t + \Delta t) - X(t)))}{d \log(\Delta t)}. \quad (32)$$

Este método es útil en series temporales no estacionarias, donde las tendencias pueden afectar el cálculo del rango reescalado.

2.4.6 Análisis de Fluctuaciones Detrendizadas (DFA)

Para corregir efectos de tendencias en la serie temporal, se usa el método DFA. Se define la función de fluctuación $F(n)$:

$$F(n) = \sqrt{\frac{1}{N} \sum_{k=1}^N [Y(k) - Y_{\text{local}}(k)]^2}, \quad (33)$$

donde $Y(k)$ es el perfil acumulado de la serie y $Y_{\text{local}}(k)$ es un ajuste local dentro de cada ventana. La relación de escalamiento es:

$$F(n) \sim n^H. \quad (34)$$

El exponente H se obtiene de la pendiente del ajuste lineal en un gráfico $\log F(n)$ contra $\log n$.

2.4.7 Relación con la Función de Autocorrelación

La función de autocorrelación de un proceso fraccional sigue:

$$C(\tau) \sim \tau^{2H-2}. \quad (35)$$

Esto implica que:

- Si $H > 0.5$, la autocorrelación decae lentamente, indicando dependencia a largo plazo.
- Si $H < 0.5$, la autocorrelación es negativa, lo que sugiere reversión a la media.
- Si $H = 0.5$, el proceso es un ruido blanco sin memoria.

2.4.8 Aplicaciones del Exponente de Hurst

El exponente de Hurst se usa en múltiples disciplinas:

- Finanzas: Identificación de tendencias en los mercados.
- Física estadística: Modelado de sistemas fuera del equilibrio.
- Climatología: Análisis de patrones climáticos de largo plazo.
- Biología: Modelado de dinámicas de poblaciones y ritmos fisiológicos.

2.5 Teoría de la Volatilidad

[7] Cálculo de la Volatilidad en Series Temporales en Econofísica

En econofísica, la volatilidad se utiliza para medir la variabilidad de los retornos financieros en una serie temporal. Es una herramienta clave para cuantificar el riesgo de un activo financiero.

2.5.1 Definición Matemática

La volatilidad σ se define como la desviación estándar de los retornos logarítmicos en un período determinado:

$$\sigma = \sqrt{\frac{1}{N-1} \sum_{t=1}^N (r_t - \bar{r})^2} \quad (36)$$

donde:

- r_t representa el retorno logarítmico en el instante t .

- $\bar{r}$ es el retorno promedio de la serie temporal.
- N es el número total de observaciones en la ventana de tiempo considerada.

2.5.2 Cálculo en Series de Tiempo Financieras

Dado un conjunto de precios de cierre P_t de un activo financiero, los retornos logarítmicos se definen como:

$$r_t = \log \frac{P_t}{P_{t-1}} \quad (37)$$

Luego, la volatilidad puede calcularse en una ventana de tiempo deslizante para analizar su evolución en el tiempo:

$$\sigma_T = \sqrt{\frac{1}{T-1} \sum_{t=1}^T (r_t - \bar{r})^2} \quad (38)$$

donde T es el tamaño de la ventana temporal.

2.5.3 Interpretación de la Volatilidad

La volatilidad proporciona información clave sobre la estabilidad de un mercado financiero:

- **Alta volatilidad:** Indica grandes fluctuaciones en los precios, lo que sugiere un mercado inestable y riesgoso.
- **Baja volatilidad:** Señal de un mercado más estable, con movimientos de precios más predecibles.
- **Volatilidad estacional:** Puede reflejar patrones recurrentes en los mercados, como eventos macroeconómicos o efectos de calendario.

3 Planteamiento del Problema

El problema central es evaluar la predictibilidad del S&P500 mediante una red neuronal del tipo LSTM, usando python y mas específicamente su

librería tensorflow, entre otras mencionadas mas adelante; la predictibilidad que se logre respecto a los datos del histórico de precios de 34 años del S&P500 (1 de enero de 1990 hasta el 1 de enero de 2024, de los cuales se usaran el 80% de los datos para entrenar la red neuronal y el 20% restante para corroborar la predicción hecha por la red neuronal) de esta manera se mostrara que tan factible y preciso es lograr algo así con los recursos a disposición (el entorno de google colab y por otro lado una PC con una tarjeta gráfica 4070), también se intentara calcular de manera teórica en python, parámetros como la entropía de Shanon; para analizar en el tiempo en que medida cambia la diversificación de estados posibles del S&P500, la fractalidad (análisis de dimensión fractal y su relación con el exponente de Hurst) lo cual no ayudara a corroborar que tan cerca esta el S&P500 de ser un mercado eficiente y finalmente la volatilidad,

4 Objetivos

4.1 Objetivo General

Desarrollar un modelo basado en redes neuronales para predecir el precio del S&P500 (teniendo en cuenta los datos del histórico de precios desde el 1 de enero de 1990 hasta el 1 de enero de 2024) con: 252 días de negociación en la bolsa de valores de EE.UU y como son 34 años, lo que da un total de : **8568** datos, utilizando el modelo de red neuronal LSTNM en python inspirándonos de [2] para ademas intentar con la misma red neuronal predecir el comportamiento de parámetros muy importantes para la econofísica como: la entropía de Shanon, el índice de Hurst (y su relación con la fractalidad) y la volatilidad; aplicando en un primer caso la normalización de todos los datos obtenidos de yahoo finance del S&P500 y después de la normalización, la separación de los datos en todos los casos, de todas las redes tipo LSTM, para todos los parámetros en un 80% para entrenamiento y un 20% para comparación, en este primer caso hay fuga de infor-

mación, ya que por la normalización inicial sin separación de los datos, la red aprende comportamientos del 20% de los datos con los que luego será comparado, haciendo que la predicción le sea mas sencilla; y en un segundo caso: primero la separación de los datos de 80% para entrenamiento y 20% para comparación, y luego la normalización, para de esta manera evitar la fuga de información y que la red no aprenda comportamientos del 20% de los datos con los que sera comparada, y de esta manera su predicción no tenga ayudas; para posteriormente comparar todas las predicciones no solo con los valores calculados teóricamente (usando parámetros los R^2 , MAE , MSE , $RMSE$), si no también comparar si las fugas de información a causa de la previa normalización afectan en que medida la calidad de las predicciones hechas por las redes tipo LSTM; finalmente también poder llegar a concluir si el equipo de hardware y software usados son lo suficientemente capaces de un objetivo como este y de no ser así explicar por que no o por que ciertas partes si o tras no de los objetivos son posibles o no.

4.2 Objetivos Específicos

1. **Objetivo Especifico Analítico:** Desarrollar un análisis analítico, desde el calculo tensorial sobre el no desvanecimiento del gradiente, su propagación y optimización, para lograr la memoria a largo plazo en la red tipo LSTM, ademas de obtener también desde el calculo tensorial las expresiones que definen la puertas lógicas (puerta de olvido, puerta de entrada y puerta de salida) dentro de cada celda que componen la cadena de la red neuronal tipo LSTM, las cuales son de vital importancia para la teoría que sustenta las redes neuronales de tipo LSTM y que le otorgan su gran ventaja de memoria graduable a largo plazo frente a las demás tipos de redes neuronales.
2. Probar el modelo LSTM para predecir el comportamiento de los precios del S&P500 de una manera básica pero representativa, donde primero se hizo la normalización previa a la separación de los datos en 80% para entrenamiento y el 20% para comparar, de tal manera que halla fuga de información, y la red pueda saber un poco el comportamiento del 20% de los datos con los que luego sera comparada su predicción, dándole esto un poco de ventaja en la preducción, para luego realizar lo mismo pero primero separando los datos en sus 80% y 20% correspondiente, para luego normalizar y que de esta menta no halla fuga de información y la red LSTM no tenga nada de información del 20% de losdatos con el que su predicción será comprada; de esta manera plantar la base de lo ya hecho por otros y partir de ahí para desarrollar los objetivos siguientes y mas importantes, esto se hará en el entorno de Google colab para saber su rendimiento, y finalmente determinar si es una herramienta suficiente en velocidad para el objetivo.
3. Plantear un código en python en el entorno de Google colab que calcule de manera teórica la serie de variación en el tiempo del indice de Hurst, para el historico de precio del S&P500 desde el 1 de enero de 1990 hasta el 1 de enero de 2024, y con el 80% de esa serie de variación del indice de Hurst, alimentar una red neuronal tipo LSTM y hacer que esta red neuronal prediga el comportamiento del ultimo 20% de la serie del indice de Hurst. 'Lo cual se hará de las dos maneras planteadas en el objetivo especifico 2: con normalización que causa fuga de información y con normalización que NO causa fuga de información, esto se hará en el entorno de Google colab para saber su rendimiento, y finalmente determinar si es una herramienta suficiente en velocidad para el objetivo.
4. Plantear un código en python en el entorno de Google colab que calcule de manera teórica la serie de variación en el tiempo de la entropía de Shannon, para el historico de

precio del S&P500 desde el 1 de enero de 1990 hasta el 1 de enero de 2024, y con el 80% de esa serie de variación de la entropía de Shannon, alimentar una red neuronal tipo LSTM y hacer que esta red neuronal prediga el comportamiento del ultimo 20% de la serie de la entropía de Shannon. 'Lo cual se hará de las dos maneras planteadas en el objetivo específico 2: con normalización que causa fuga de información y con normalización que NO causa fuga de información, esto se hará en el entorno de Google colab para saber su rendimiento, y finalmente determinar si es una herramienta suficiente en velocidad para el objetivo.

5. Plantear un código en python en el entorno de Google colab que calcule de manera teórica la serie de variación en el tiempo de la fractalidad, para el historico de precio del S&P500 desde el 1 de enero de 1990 hasta el 1 de enero de 2024, y con el 80% de esa serie de variación de la fractalidad, alimentar una red neuronal tipo LSTM y hacer que esta red neuronal prediga el comportamiento del ultimo 20% de la serie de la fractalidad. 'Lo cual se hará de las dos maneras planteadas en el objetivo específico 2: con normalización que causa fuga de información y con normalización que NO causa fuga de información, esto se hará en el entorno de Google colab para saber su rendimiento, y finalmente determinar si es una herramienta suficiente en velocidad para el objetivo.
6. Plantear un código en python en el entorno de Google colab que calcule de manera teórica la serie de variación en el tiempo de la volatilidad, para el historico de precio del S&P500 desde el 1 de enero de 1990 hasta el 1 de enero de 2024, y con el 80% de esa serie de variación de la volatilidad, alimentar una red neuronal tipo LSTM y hacer que esta red neuronal prediga el comportamiento del ultimo 20% de la serie de la volatilidad. 'Lo cual se hará de las dos maneras planteadas en el objetivo específico 2: con normal-

ización que causa fuga de información y con normalización que NO causa fuga de información, esto se hará en el entorno de Google colab para saber su rendimiento, y finalmente determinar si es una herramienta suficiente en velocidad para el objetivo.

5 Metodología

5.1 Metodología del primer objetivo específico

La metodología utilizada en el análisis consistió en (1) *definir* las ecuaciones fundamentales de la celda LSTM (puertas de entrada, olvido, salida y estado de memoria) basadas en la formulación clásica de σ y $\tanh$; (2) *demostrar matemáticamente* a partir del calculo tensorial, como se forman y la manera en que dichas puertas permiten un flujo de gradiente estable, mostrando con el calculo tensorial la propagación y optimización del gradiente, examinando en detalle la derivada $\partial c_t / \partial c_{t-1}$ y resaltando el papel de la puerta de olvido f_t ; y (3) *discutir* la relevancia de estas ecuaciones para evitar el desvanecimiento del gradiente en el contexto de la econofísica, donde las dependencias a largo plazo son cruciales para analizar crisis y correlaciones financieras. Este procedimiento se sustentó en los trabajos pioneros de [1], quienes introdujeron la arquitectura LSTM, y en la referencia de [11], que detalla la teoría y práctica del aprendizaje profundo, incluidas las redes recurrentes avanzadas.

5.2 Metodología y explicación del código para segundo objetivo específico

Se partirá usando el entorno de desarrollo de código de Google; llamado Google colab, el cual vía internet brinda un espacio en un servidor con 2 gigas de RAM y un motor de procesamiento de la compañía tesla, el cual no es muy actual, esta elección se hace para partir de los menores

recursos posibles para saber si son viables o no para este objetivo en específico, el cual consiste en plantear un código en python que cree una red neuronal tipo LSTM, la cual en primer caso sera alimentada con el 80% del histórico de precios del S&P500 desde el 1 de enero de 1990 hasta el 1 de enero de 2024, para esto vamos a tomar la información de [9] para no tener que descargar tantos datos, pero esto exige tener conexión estable a internet, donde se realizo la normalización del 100% de los datos, antes de separarlos en el 80% de los datos para entrenamiento y el 20% de los datos para comparación, al normalizar primero habrá fuga de datos que hará que la red LSTM aprenda comportamientos del 20% de los datos con los que luego será comparada su predicción; después se hizo el mismo procedimiento pero la normalización se realizo después de separar los datos en el 80% y el 20% correspondiente, corrigiendo así la fuga de información, en ambos casos después de cual halla sido la normalización, definimos la red neuronal tipo LSTM, compilamos el modelo, entrenamos el modelo en la GPU, hacemos la predicciones y finalmente graficamos los resultados (ver celda 1 y 2 de [8]).

Código de Predicción del S&P 500 con LSTM

El código implementa una red neuronal LSTM para predecir los precios del S&P 500 utilizando datos históricos descargados desde Yahoo Finance. Se aplican técnicas de preprocesamiento, modelado y evaluación para optimizar la predicción. A continuación, se explica cada paso matemáticamente.

Descarga y Preprocesamiento de Datos

El código utiliza Yahoo Finance para obtener los precios de cierre del índice S&P 500 desde el 1 de enero de 1990 hasta el 1 de enero de 2024. Se normalizan los datos para evitar la fuga de información (en el primer caso se normaliza di-

rectamente todos los datos) mediante la transformación min-max:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (39)$$

donde x es el valor original, y $x_{\min}$, $x_{\max}$ son los valores mínimo y máximo del conjunto de entrenamiento.

Creación de Secuencias para Entrenamiento

Se construyen secuencias de entrada y salida con longitud L para alimentar la red:

$$\mathbf{X}_t = [x_t, x_{t-1}, \dots, x_{t-L+1}]^T \quad (40)$$

$$y_t = x_{t+1} \quad (41)$$

donde $\mathbf{X}_t$ es la secuencia de entrada y y_t es la salida esperada.

Arquitectura de la Red LSTM

El modelo LSTM se define con las siguientes capas:

- **Capa LSTM 1:** Con 50 neuronas y activación de función tanh.
- **Dropout:** Reduce sobreajuste eliminando conexiones con probabilidad $p = 0.2$.
- **Capa LSTM 2:** Similar a la primera pero sin retorno de secuencias.
- **Capa Dense:** Con 25 neuronas y activación ReLU.
- **Capa Dense de salida:** Una neurona sin activación.

Ecuaciones de la Celda LSTM

La celda LSTM está definida por:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f) \quad (42)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i) \quad (43)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c) \quad (44)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (45)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o) \quad (46)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (47)$$

Entrenamiento del Modelo

Se usa la función de pérdida de error cuadrático medio:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (48)$$

donde y_i es el valor real y $\hat{y}_i$ es la predicción. El optimizador Adam ajusta los pesos usando:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (49)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (50)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (51)$$

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (52)$$

con $\beta_1 = 0.9$, $\beta_2 = 0.999$ y $\epsilon = 10^{-8}$.

Evaluación del Modelo

Se usan métricas de error:

- **Coefficiente R^2 :**

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} \quad (53)$$

- **Error porcentual absoluto medio (MAPE):**

$$MAPE = \frac{100}{N} \sum_{i=1}^N \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (54)$$

Resultados

La predicción se grafica comparando valores reales y estimados:

$$\text{plot}(\text{fecha}, y_{\text{real}}, \text{color} = \text{"blue"}) \quad (55)$$

$$\text{plot}(\text{fecha}, y_{\text{pred}}, \text{color} = \text{"red"}) \quad (56)$$

El modelo muestra buen ajuste si R^2 se acerca a 1 y MAPE es bajo.

5.3 Metodología y explicación del código para el tercer objetivo específico

La metodología seguida en el código inicia con la *descarga de datos* reales del índice S&P 500 mediante la librería `yfinance`, el cual consiste en plantear un código en python que cree una red neuronal tipo LSTM, la cual en primer caso sera alimentada con el 80% del histórico de precios del S&P500 desde el 1 de enero de 1990 hasta el 1 de enero de 2024, para esto vamos a tomar la información de [9] para no tener que descargar tantos datos, pero esto exige tener conexión estable a internet, donde se realizo la normalización del 100% de los datos, antes de separarlos en el 80% de los datos para entrenamiento y el 20% de los datos para comparación, al normalizar primero habrá fuga de datos que hará que la red LSTM aprenda comportamientos del 20% de los datos con los que luego será comparada su predicción; después se hizo el mismo procedimiento pero la normalización se realizo después de separar los datos en el 80% y el 20% correspondiente, corrigiendo así la fuga de información, en ambos casos después de cual halla sido la normalización, tras lo cual se *calcula* el retorno logarítmico diario para generar una serie estacionaria. A continuación, se *implementa* el método R/S para estimar el exponente de Hurst en ventanas móviles, lo que permite obtener una serie temporal de valores de Hurst. Seguidamente, dicha serie se *normaliza* utilizando `MinMaxScaler` y se *transforma* en secuencias para alimentar una red neuronal de tipo LSTM. La red se *entrena* sobre

el conjunto de entrenamiento, monitorizando su desempeño en el conjunto de validación, y finalmente se *evalúa* el modelo comparando las predicciones con los valores reales de Hurst en el conjunto de prueba. Con ello, se obtiene una perspectiva tanto del valor teórico del exponente de Hurst como de la predicción ofrecida por la red LSTM, para ver el código en cuestión (ver la celda 6 y 7 de [8]).

Código de predicción del índice de Hurst para el S&P500

Cálculo del Índice de Hurst

El índice de Hurst se estima usando el método R/S, donde la relación rescaled range se define como:

$$H = \lim_{m \rightarrow \infty} \frac{\log(R(m)/S(m))}{\log m} \quad (57)$$

donde $R(m)$ es el rango de la suma acumulada centrada y $S(m)$ es la desviación estándar en la ventana de tamaño m .

Cálculo de R/S en Ventanas Móviles

Para cada ventana de tamaño m , se calcula:

$$Y_i = \sum_{j=1}^i (X_j - \bar{X}) \quad \text{para } i = 1, 2, \dots, m \quad (58)$$

$$R(m) = \max(Y) - \min(Y) \quad (59)$$

$$S(m) = \sqrt{\frac{1}{m} \sum_{i=1}^m (X_i - \bar{X})^2} \quad (60)$$

donde X_j son los retornos logarítmicos y $\bar{X}$ es su media.

Descarga y Preprocesamiento de Datos

Los precios de cierre del S&P 500 se descargan desde Yahoo Finance y se transforman en re-

tornos logarítmicos:

$$R_t = \log P_t - \log P_{t-1} \quad (61)$$

donde P_t es el precio de cierre en el tiempo t .

Preparación de Datos para la Red LSTM

Los valores de Hurst se normalizan usando la transformación Min-Max para que no haya fuga de datos, para el primer caso el código es el mismo solo que la normalización se hace directa:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (62)$$

Luego, se crean secuencias de entrada y salida:

$$\mathbf{X}_t = [H_t, H_{t-1}, \dots, H_{t-L+1}]^T \quad (63)$$

$$y_t = H_{t+1} \quad (64)$$

donde L es la longitud de la secuencia.

Arquitectura de la Red LSTM

La red LSTM utilizada consta de:

- Una capa LSTM con 50 neuronas y activación tanh.
- Una capa densa con una neurona de salida.

Las ecuaciones de la celda LSTM son:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f) \quad (65)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i) \quad (66)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c) \quad (67)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (68)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o) \quad (69)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (70)$$

Entrenamiento del Modelo

Se minimiza el error cuadrático medio:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (71)$$

El optimizador Adam ajusta los pesos mediante:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (72)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (73)$$

$$\theta_t = \theta_{t-1} - \alpha \frac{m_t}{\sqrt{v_t} + \epsilon} \quad (74)$$

donde $\beta_1 = 0.9$, $\beta_2 = 0.999$ y $\epsilon = 10^{-8}$.

Evaluación del Modelo

Las predicciones son evaluadas con:

- Error absoluto medio (MAE):

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \quad (75)$$

- Error cuadrático medio (MSE):

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (76)$$

- Raíz del error cuadrático medio (RMSE):

$$RMSE = \sqrt{MSE} \quad (77)$$

- Coeficiente R^2 :

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} \quad (78)$$

Resultados

El exponente de Hurst teórico se calcula como:

$$H_{\text{teórico}} = \frac{1}{N} \sum_{i=1}^N H_i \quad (79)$$

y el valor predicho por la red LSTM es:

$$H_{\text{predicho}} = \frac{1}{M} \sum_{i=1}^M \hat{H}_i \quad (80)$$

5.4 Metodología y explicación del código para cuarto objetivo específico

En este código se inicia con la *descarga de datos* del S&P500 a través de `yfinance`, de donde se obtiene el precio de cierre. Posteriormente, se *calcula* el retorno logarítmico diario y se aplica una *ventana móvil* para estimar la *entropía de Shannon* sobre los datos de cada ventana, lo que genera una serie temporal de entropías. A continuación, dicha serie se *normaliza* con `MinMaxScaler` y se *transforma* en secuencias para alimentar una red neuronal LSTM. El modelo se *entrena* con un conjunto de entrenamiento y se *valida* con el conjunto de prueba, tras lo cual se *grafica* la entropía teórica frente a la predicción de la red. Con ello, se evalúa la capacidad de la LSTM para modelar la evolución temporal de la entropía de Shannon en la serie de retornos del S&P500, para ver el código en cuestión (ver la celda 8 y 9 de [8]).

Código de predicción de la Entropía de Shannon para el S&P500

El código implementa una función para calcular la entropía de Shannon a partir de una muestra de datos utilizando la siguiente fórmula:

$$H(X) = - \sum p_i \log_2(p_i) \quad (81)$$

donde p_i representa las probabilidades de los valores dentro de la muestra. Se usa una histogramación de 30 bins para calcular estas probabilidades.

Descarga de Datos del S&P 500

Se extraen los datos históricos del índice S&P 500 utilizando la librería `yfinance`. Posterior-

mente, se calculan los retornos logarítmicos:

$$r_t = \log \left(\frac{P_t}{P_{t-1}} \right) \quad (82)$$

Cálculo de la Entropía en Ventanas Móviles

Se aplica una ventana deslizante de 252 días para calcular la entropía de Shannon en cada ventana de datos. Este enfoque permite analizar la evolución de la entropía en el tiempo.

Preparación de los Datos para la LSTM

Se normalizan los datos utilizando la transformación Min-Max para evitar fuga de información (en el primer caso se normaliza directamente), tal como se muestra en las descripciones de los códigos de las metodologías anteriores. Luego, se generan secuencias de entrada con una longitud de 30 días para predecir la entropía futura.

Construcción del Modelo LSTM

Se define una red neuronal con una capa LSTM de 50 neuronas y una capa densa final. Se utiliza la función de activación ReLU en la LSTM y se compila con el optimizador Adam y la función de pérdida de error cuadrático medio.

Entrenamiento y Evaluación del Modelo

El modelo se entrena durante 50 épocas con lotes de tamaño 32. Luego, se realizan predicciones sobre el conjunto de prueba y se evalúan las métricas de error:

- MAE: Error absoluto medio.
- MSE: Error cuadrático medio.
- RMSE: Raíz del error cuadrático medio.
- R² Score: Coeficiente de determinación.
- MAPE: Error porcentual absoluto medio.

7. Visualización de Resultados

Finalmente, se grafican los valores reales de la entropía y las predicciones generadas por la red LSTM.

5.5 Metodología y explicación del código para quinto objetivo específico

En este código se inicia con la *descarga de datos* del S&P500 a través de `yfinance`, obteniendo el precio de cierre. Luego, se *calcula* el retorno logarítmico diario y se aplica una *ventana móvil* para estimar la *dimensión fractal* mediante el método de Higuchi, generando así una serie temporal de valores de fractalidad. Posteriormente, la serie se *normaliza* utilizando `MinMaxScaler` y se *convierte* en secuencias de datos para alimentar una red neuronal LSTM. El modelo es *entrenado* con el conjunto de entrenamiento y *validado* con el conjunto de prueba. Finalmente, se *grafica* la evolución de la fractalidad teórica junto con la predicción de la LSTM, permitiendo evaluar su capacidad para modelar la estructura fractal de los retornos del S&P500, para ver el código en cuestión (ver la celda 10 y 11 de [8]).

6 Código de predicción de la Fractalidad del S&P500

Este código implementa el cálculo de la *dimensión fractal* de una serie de retornos financieros utilizando el método de *Higuchi* y emplea una *red neuronal LSTM* para predecir su evolución temporal. A continuación, se detallan sus secciones principales:

Cálculo de la Fractalidad con el Método de Higuchi

La función `higuchi_fd(x, kmax=10)` implementa el *método de Higuchi* para estimar la dimensión fractal de una serie temporal.

- **División en segmentos:** Se divide la serie en segmentos de tamaño k .
- **Cálculo de la longitud promedio:** Se calcula la longitud promedio de cada segmento en función de k .
- **Ajuste lineal:** Se ajusta una regresión lineal entre $\ln(1/k)$ y $\ln(L_k)$ para obtener la pendiente, que representa la dimensión fractal.
- **50 neuronas** en la capa LSTM.
- **Función de activación ReLU.**
- **Una capa densa de salida** con activación lineal.
- **Optimización con Adam y pérdida de error cuadrático medio.**

El modelo se entrena por **50 épocas** con un tamaño de batch de **32**.

El resultado final es un número entre 1 y 2, donde valores más cercanos a 2 indican mayor complejidad fractal en la serie.

Descarga de Datos y Cálculo de Retornos

El código descarga los precios de cierre del **S&P 500** desde 1990 hasta 2024 utilizando `yfinance`. Luego, se calculan los *retornos logarítmicos* según la ecuación:

$$r_t = \ln \left(\frac{P_t}{P_{t-1}} \right) \quad (83)$$

Estos retornos son la base para calcular la fractalidad en ventanas de **252 días** (aproximadamente un año de datos bursátiles).

Creación de la Serie Temporal de Fractalidad

Se calcula la **dimensión fractal** en cada ventana móvil de 252 días y se guarda en `fd_series`.

Para evitar problemas de normalización con la LSTM, la serie se **divide** en conjunto de **entrenamiento (80%)** y **prueba (20%)** antes de aplicar `MinMaxScaler`, asegurando que la información del conjunto de prueba no influya en la normalización.

Construcción y Entrenamiento de la Red LSTM

La LSTM se diseña con:

Evaluación y Predicción

La LSTM realiza predicciones sobre el conjunto de prueba, y se invierte la normalización para comparar con la fractalidad real. Se calculan métricas de evaluación como:

- **Error Absoluto Medio (MAE)**
- **Error Cuadrático Medio (MSE, RMSE)**
- **Coefficiente de Determinación R^2**
- **Error Porcentual Absoluto Medio (MAPE)**

Valores de **RMSE bajos** y **R^2 cercano a 1** indicarían una buena capacidad predictiva.

Visualización de Resultados

Finalmente, se **grafica** la evolución de la fractalidad real vs. la predicción de la LSTM. Esto permite evaluar si la red neuronal logra capturar la estructura fractal de los retornos del S&P 500.

Conclusión

El código implementa una estrategia robusta para analizar la **complejidad fractal** del mercado financiero y explorar su predictibilidad con redes LSTM. Si la LSTM logra capturar patrones fractales, podría ser útil para modelar y anticipar la dinámica de los mercados bursátiles.

6.1 Metodología y explicación del código para sexto objetivo específico

En este código se inicia con la *descarga de datos* del S&P500 a través de `yfinance`, obteniendo los precios de cierre ajustados. A partir de estos datos, se calcula el *retorno logarítmico diario*, que luego se utiliza para estimar la *volatilidad histórica*. La volatilidad se calcula en *ventanas móviles* de 60 días, permitiendo obtener una serie temporal de volatilidades. Posteriormente, la serie de retornos se *normaliza* utilizando `MinMaxScaler`, asegurando que la información del conjunto de prueba no influya en el entrenamiento. Los datos son transformados en secuencias de entrada y salida para alimentar una red neuronal LSTM, la cual es diseñada con capas de 128 y 64 neuronas, además de una capa *Dropout* para reducir el sobreajuste. El modelo se *entrena* utilizando `adam` como optimizador y la función de pérdida *MSE*, validando su desempeño con un conjunto de prueba. Finalmente, se *grafica* la volatilidad real frente a la predicción de la LSTM, evaluando su capacidad para modelar la evolución temporal de la volatilidad del S&P500 mediante métricas como el error absoluto medio (MAE), el error cuadrático medio (MSE, RMSE), el coeficiente de determinación R^2 y el error porcentual absoluto medio (MAPE), para ver el código en cuestión (ver la celda 12 y 13 de [8]).

Código de Predicción de Volatilidad para el S&P500

Este código implementa un modelo basado en redes neuronales LSTM para predecir la *volatilidad histórica* del S&P500. A continuación, se detallan sus secciones principales:

6.2 Descarga de Datos y Cálculo de Retornos

El código descarga los **precios de cierre ajustados** del S&P500 utilizando la librería

`yfinance`, con un rango de datos entre 1994 y 2024.

A partir de estos precios, se calculan los *rendimientos logarítmicos* con la ecuación:

$$r_t = \ln \left(\frac{P_t}{P_{t-1}} \right) \quad (84)$$

donde P_t representa el precio en el tiempo t .

6.3 Cálculo de la Volatilidad Histórica

La volatilidad histórica se estima a partir de la desviación estándar de los rendimientos en una ventana móvil de 60 días:

$$\sigma_t = \sqrt{\frac{1}{n} \sum_{i=1}^n (r_i - \bar{r})^2} \times \sqrt{252} \quad (85)$$

donde $n = 60$ representa el tamaño de la ventana y la multiplicación por $\sqrt{252}$ ajusta la volatilidad a escala anual.

6.4 Preprocesamiento de Datos

Los datos se **normalizan** con `MinMaxScaler` en un rango de $[-1, 1]$ para los rendimientos y $[0, 1]$ para la volatilidad, asegurando que la normalización solo se ajuste con el conjunto de entrenamiento para evitar *data leakage* o fuga de información (en la primera parte de la volatilidad se hizo todo igual excepto la normalización la cual fue directa).

Se generan secuencias de entrada y salida donde:

- Cada entrada consiste en una ventana de 60 días de retornos normalizados.
- La salida es la volatilidad histórica correspondiente al último día de la ventana.

6.5 Construcción de la Red Neuronal LSTM

El modelo de red neuronal se compone de las siguientes capas:

- **Capa LSTM con 128 neuronas** y activación ReLU, que captura dependencias temporales.
- **Dropout del 20%**, para reducir el sobreajuste.
- **Segunda capa LSTM con 64 neuronas**, mejorando la abstracción de patrones.
- **Capa densa de 32 neuronas** con ReLU.
- **Capa de salida con activación lineal** para la predicción de la volatilidad.

El modelo es compilado utilizando el optimizador **Adam** y la función de pérdida **MSE**.

6.6 Entrenamiento y Validación

El modelo se entrena con:

- **80% de los datos** para entrenamiento y **20%** para validación.
- **100 épocas** y un tamaño de batch de **32**.
- **Evaluación en el conjunto de prueba** con las métricas estándar.

6.7 Evaluación del Modelo

Para evaluar el desempeño del modelo, se utilizan las siguientes métricas:

- **Error Absoluto Medio (MAE)**
- **Error Cuadrático Medio (MSE, RMSE)**
- **Coefficiente de Determinación R^2**
- **Error Porcentual Absoluto Medio (MAPE)**

Un valor de **RMSE bajo** y un **R^2 cercano a 1** indicarían que el modelo es capaz de capturar correctamente la dinámica de la volatilidad.

6.8 Visualización de Resultados

Finalmente, se grafican la **volatilidad real** y la **volatilidad predicha** para evaluar visualmente el desempeño del modelo.

6.9 Conclusión

El código implementa una metodología basada en LSTM para modelar la evolución temporal de la **volatilidad del S&P500**. Si el modelo logra capturar los patrones de la volatilidad, podría ser útil en aplicaciones de *gestión de riesgos* y *estrategias de inversión*.

7 Recursos Disponibles

Se utilizarán bibliotecas de Python como TensorFlow, NumPy y Pandas para emplear el modelo LSTM. Primeramente el intento en poder desarrollar todo en el entorno de Google colab (vamos a ver que tan lejos puede llegar un estudiante de física de la UNAL con acceso a internet para predecir parámetros importantes del S&P500), para después también usar un PC con hardware con soporte CUDA, procesador de última generación de intel, tarjeta gráfica 4070 de Nvidia y 32 gigas de RAM; para acelerar el entrenamiento de la red neuronal.

8 Desarrollo de los objetivos específicos

8.1 Desarrollo del primer objetivo: Objetivo específico analítico

[12] Las redes neuronales de memoria a largo plazo (LSTM) mejoran a las redes neuronales recurrentes (RNN) mediante una estructura de memoria controlada por puertas. En este documento, se desarrolla matemáticamente la primera puerta lógica de una celda LSTM, la **puerta de olvido**, desde el cálculo tensorial hasta su optimización.

Fundamentos de Cálculo Tensorial en Redes Neuronales

[11] En el contexto de redes neuronales, los tensores se definen de la siguiente manera:

- $\mathbf{x}_t \in R^d$ es la entrada en el tiempo t .
- $\mathbf{h}_{t-1} \in R^h$ es el estado oculto previo.
- $\mathbf{c}_{t-1} \in R^h$ es el estado de memoria previo.
- $\mathbf{W}_f \in R^{h \times d}$, $\mathbf{U}_f \in R^{h \times h}$ son matrices de pesos.
- $\mathbf{b}_f \in R^h$ es un vector de sesgo.

Cada capa de una red neuronal transforma la entrada $\mathbf{x}_t$ y el estado oculto previo $\mathbf{h}_{t-1}$ mediante una transformación afín seguida de una activación:

$$\mathbf{z}_t = \mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b} \quad (86)$$

donde $\mathbf{z} \in R^{4h}$ contiene las activaciones de todas las puertas:

$$\mathbf{z} = \begin{bmatrix} \mathbf{z}_f \\ \mathbf{z}_i \\ \mathbf{z}_o \\ \mathbf{z}_c \end{bmatrix} = \begin{bmatrix} \mathbf{W}_f\mathbf{x}_t + \mathbf{U}_f\mathbf{h}_{t-1} + \mathbf{b}_f \\ \mathbf{W}_i\mathbf{x}_t + \mathbf{U}_i\mathbf{h}_{t-1} + \mathbf{b}_i \\ \mathbf{W}_o\mathbf{x}_t + \mathbf{U}_o\mathbf{h}_{t-1} + \mathbf{b}_o \\ \mathbf{W}_c\mathbf{x}_t + \mathbf{U}_c\mathbf{h}_{t-1} + \mathbf{b}_c \end{bmatrix} \quad (87)$$

Transformación de Tensores en Funciones de Activación

Cada componente de $\mathbf{z}$ se somete a una función de activación:

$$\mathbf{f}_t = \sigma(\mathbf{z}_f) = \sigma(\mathbf{W}_f\mathbf{x}_t + \mathbf{U}_f\mathbf{h}_{t-1} + \mathbf{b}_f) \quad (88)$$

$$\mathbf{i}_t = \sigma(\mathbf{z}_i) = \sigma(\mathbf{W}_i\mathbf{x}_t + \mathbf{U}_i\mathbf{h}_{t-1} + \mathbf{b}_i) \quad (89)$$

$$\mathbf{o}_t = \sigma(\mathbf{z}_o) = \sigma(\mathbf{W}_o\mathbf{x}_t + \mathbf{U}_o\mathbf{h}_{t-1} + \mathbf{b}_o) \quad (90)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{z}_c) = \tanh(\mathbf{W}_c\mathbf{x}_t + \mathbf{U}_c\mathbf{h}_{t-1} + \mathbf{b}_c) \quad (91)$$

donde $\sigma(x) = \frac{1}{1+e^{-x}}$ y $\tanh(x)$ regulan los valores.

Construcción de la Ecuación de Estado en LSTM

Las puertas actualizan el estado de memoria y salida:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (92)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (93)$$

donde $\odot$ denota el producto elemento a elemento.

Aplicación como Puerta Lógica en LSTM

Cada puerta en LSTM actúa como un interruptor suave:

- Si $\mathbf{f}_t \approx 1$, la memoria se conserva.
- Si $\mathbf{f}_t \approx 0$, la memoria se olvida.
- Si $\mathbf{i}_t \approx 1$, se introduce nueva memoria.
- Si $\mathbf{o}_t \approx 1$, la salida refleja la memoria.

Este diseño mitiga el desvanecimiento del gradiente en RNNs estándar.

Definición de la Puerta de Olvido

La puerta de olvido en una celda LSTM se define como:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f\mathbf{x}_t + \mathbf{U}_f\mathbf{h}_{t-1} + \mathbf{b}_f) \quad (94)$$

donde $\sigma(x)$ es la función sigmoide:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (95)$$

El estado de memoria se actualiza como:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (96)$$

donde:

- $\mathbf{i}_t$ es la puerta de entrada.

- $\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c)$ es la nueva información candidata.

- $\odot$ representa el producto elemento a elemento (Hadamard).

Si $\mathbf{f}_t$ se aproxima a 1, se conserva la memoria $\mathbf{c}_{t-1}$. Si $\mathbf{f}_t$ se aproxima a 0, la memoria se olvida.

Propagación del Gradiente y Optimización

Para entrenar la celda LSTM, usamos el método de retropropagación a través del tiempo (BPTT). La pérdida L depende de la salida final h_t , y sus gradientes se propagan hasta la puerta de olvido.

El gradiente de la pérdida respecto a la puerta de olvido es:

$$\frac{\partial L}{\partial \mathbf{f}_t} = \frac{\partial L}{\partial \mathbf{c}_t} \odot \mathbf{c}_{t-1} \quad (97)$$

Dado que la función sigmoide tiene derivada:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)) \quad (98)$$

el gradiente de $\mathbf{f}_t$ respecto a su entrada es:

$$\frac{\partial \mathbf{f}_t}{\partial \mathbf{z}_f} = \mathbf{f}_t \odot (1 - \mathbf{f}_t) \quad (99)$$

Aplicando la regla de la cadena:

$$\frac{\partial L}{\partial \mathbf{z}_f} = \frac{\partial L}{\partial \mathbf{f}_t} \odot \mathbf{f}_t \odot (1 - \mathbf{f}_t) \quad (100)$$

Finalmente, los gradientes para actualizar los parámetros son:

$$\frac{\partial L}{\partial \mathbf{W}_f} = \sum_t \frac{\partial L}{\partial \mathbf{z}_f} \mathbf{x}_t^T \quad (101)$$

$$\frac{\partial L}{\partial \mathbf{U}_f} = \sum_t \frac{\partial L}{\partial \mathbf{z}_f} \mathbf{h}_{t-1}^T \quad (102)$$

$$\frac{\partial L}{\partial \mathbf{b}_f} = \sum_t \frac{\partial L}{\partial \mathbf{z}_f} \quad (103)$$

Definición de la Puerta de Entrada

La puerta de entrada en una celda LSTM se define como:

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i) \quad (104)$$

donde $\sigma(x)$ es la función sigmoide:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (105)$$

La puerta de entrada decide cuánta información nueva se almacenará en la celda de memoria. La información candidata a ser almacenada se define como:

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c) \quad (106)$$

Finalmente, la actualización del estado de memoria se expresa como:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (107)$$

donde:

- $\mathbf{f}_t$ es la puerta de olvido.
- $\tilde{\mathbf{c}}_t$ es el nuevo estado candidato de la memoria.
- $\odot$ representa el producto elemento a elemento (Hadamard).

Si $\mathbf{i}_t$ se aproxima a 1, se introduce información en la memoria. Si $\mathbf{i}_t$ se aproxima a 0, la información se bloquea.

Propagación del Gradiente y Optimización

Para entrenar la celda LSTM, se emplea la retropropagación a través del tiempo (BPTT). La pérdida L depende de la salida final h_t , y sus gradientes se propagan hasta la puerta de entrada.

El gradiente de la pérdida respecto a la puerta de entrada es:

$$\frac{\partial L}{\partial \mathbf{i}_t} = \frac{\partial L}{\partial \mathbf{c}_t} \odot \tilde{\mathbf{c}}_t \quad (108)$$

Dado que la función sigmoide tiene derivada:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)) \quad (109)$$

el gradiente de $\mathbf{i}_t$ respecto a su entrada es:

$$\frac{\partial \mathbf{i}_t}{\partial \mathbf{z}_i} = \mathbf{i}_t \odot (1 - \mathbf{i}_t) \quad (110)$$

Aplicando la regla de la cadena:

$$\frac{\partial L}{\partial \mathbf{z}_i} = \frac{\partial L}{\partial \mathbf{i}_t} \odot \mathbf{i}_t \odot (1 - \mathbf{i}_t) \quad (111)$$

Finalmente, los gradientes para actualizar los parámetros son:

$$\frac{\partial L}{\partial \mathbf{W}_i} = \sum_t \frac{\partial L}{\partial \mathbf{z}_i} \mathbf{x}_t^T \quad (112)$$

$$\frac{\partial L}{\partial \mathbf{U}_i} = \sum_t \frac{\partial L}{\partial \mathbf{z}_i} \mathbf{h}_{t-1}^T \quad (113)$$

$$\frac{\partial L}{\partial \mathbf{b}_i} = \sum_t \frac{\partial L}{\partial \mathbf{z}_i} \quad (114)$$

Definición de la Puerta de Salida

La puerta de salida en una celda LSTM se define como:

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o) \quad (115)$$

donde $\sigma(x)$ es la función sigmoide:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (116)$$

La salida final de la celda LSTM está dada por:

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (117)$$

donde:

- $\mathbf{c}_t$ es el estado de memoria actualizado.

- $\tanh(\mathbf{c}_t)$ es una versión filtrada de la memoria.

- $\mathbf{o}_t$ regula qué parte de la memoria se transfiere a la salida.

- $\odot$ representa el producto elemento a elemento (Hadamard).

Si $\mathbf{o}_t$ se aproxima a 1, la memoria filtrada se transmite a la salida. Si $\mathbf{o}_t$ se aproxima a 0, la celda no emite información.

Propagación del Gradiente y Optimización

Para entrenar la celda LSTM, se emplea la retropropagación a través del tiempo (BPTT). La pérdida L depende de la salida final h_t , y sus gradientes se propagan hasta la puerta de salida.

El gradiente de la pérdida respecto a la puerta de salida es:

$$\frac{\partial L}{\partial \mathbf{o}_t} = \frac{\partial L}{\partial \mathbf{h}_t} \odot \tanh(\mathbf{c}_t) \quad (118)$$

Dado que la función sigmoide tiene derivada:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)) \quad (119)$$

el gradiente de $\mathbf{o}_t$ respecto a su entrada es:

$$\frac{\partial \mathbf{o}_t}{\partial \mathbf{z}_o} = \mathbf{o}_t \odot (1 - \mathbf{o}_t) \quad (120)$$

Aplicando la regla de la cadena:

$$\frac{\partial L}{\partial \mathbf{z}_o} = \frac{\partial L}{\partial \mathbf{o}_t} \odot \mathbf{o}_t \odot (1 - \mathbf{o}_t) \quad (121)$$

Finalmente, los gradientes para actualizar los parámetros son:

$$\frac{\partial L}{\partial \mathbf{W}_o} = \sum_t \frac{\partial L}{\partial \mathbf{z}_o} \mathbf{x}_t^T \quad (122)$$

$$\frac{\partial L}{\partial \mathbf{U}_o} = \sum_t \frac{\partial L}{\partial \mathbf{z}_o} \mathbf{h}_{t-1}^T \quad (123)$$

$$\frac{\partial L}{\partial \mathbf{b}_o} = \sum_t \frac{\partial L}{\partial \mathbf{z}_o} \quad (124)$$

8.2 Desarrollo del segundo objetivo específico

Después de aplicar lo dicho en la metodología del segundo objetivo específico, se obtuvieron resultados muy satisfactorios como se puede ver en las figuras 3 y 4, la cual indica:

Con Normalización directa y fuga de información

Se obtuvo un comportamiento muy similar en el cambio de los precios (véase figura 5), obtenido por la red LSTM, en comparación con la serie del histórico de precios del S&P500, para el tiempo entre 2017 y 2024, que es el 20% de los datos para comparación, siendo estos sus parámetros de comparación: R^2 : 0.9910 y $MAPE$: 1.58%, mostrando así que la predicción hecha por la red LSTM es muy buena ya que también tiene ayuda de la fuga de información causada por la normalización inicial del 100% de los datos.



Figure 3: Gráfica de comparación entre el histórico de precios real del S&P500 y la predicción hecha por la red neuronal LSTM, desde 2017 hasta 2024, con fuga de información.

Con normalización Max- Min y sin fuga de información

Se obtuvo una buena predicción en las tendencias por parte de la red LSTM, respecto el historico de los precios del S&P500 desde aproximadamente 2017 hasta el 2024 (que es el lapso de tiempo del 20% de los datos para la com-

paración) pero no tan buena como la realizada con fuga de información (vease figura 4) obteniendo estos parametros de comparación: R^2 : 0.9637 y $MAPE$: 2.62%

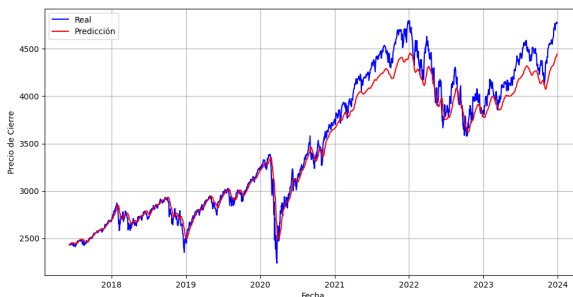


Figure 4: Gráfica de comparación entre el histórico de precios real del S&P500 y la predicción hecha por la red neuronal LSTM, desde 2017 hasta 2024, sin fuga de información.

Corroborando así lo logrado en repetidas ocasiones por las financieras que realizan análisis predictivo sobre series de tiempo del precio de activos financieros, obteniendo así la herramienta fundamental para el desarrollo de los objetivos realmente centrales de este proyecto, cabe aclarar que aunque se logro este objetivo, las modificaciones necesarias para que una red neuronal que en este objetivo logro su cometido, son bastantes y mucho mas complejas. Finalmente se muestra que con el entorno de Google colab es suficiente para conseguir el segundo objetivo específico.

8.3 Desarrollo del tercer objetivo específico

Después de aplicar lo dicho en la metodología del tercer objetivo específico, se obtuvieron resultados muy satisfactorios como se puede ver en las figuras 5 y 6, la cual indica:

Con Normalización directa y fuga de información

Se obtuvo un comportamiento muy similar en la evolución histórica del índice de Hurst (véase figura 5), obtenido por la red LSTM, en comparación con la serie del histórica obtenida teóricamente del índice de Hurst del S&P500, para el tiempo entre 2017 y 2024, que es el 20% de los datos para comparación, siendo estos sus parámetros de comparación: MAE : 0.0107, MSE : 0.0002, $RMSE$: 0.0136, R^2 : 0.9032, $MAPE$: 1.99%, mostrando así que la predicción hecha por la red LSTM es muy buena ya que también tiene ayuda de la fuga de información causada por la normalización inicial del 100% de los datos.

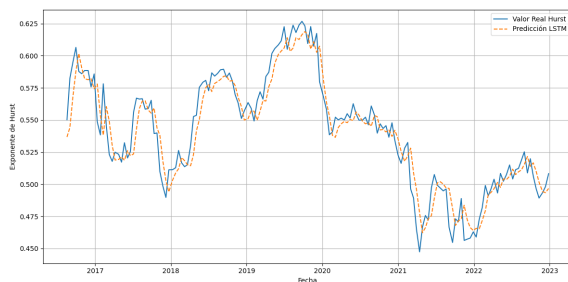


Figure 5: Gráfica de comparación entre el histórico del índice de Hurst teórico del S&P500 y la predicción hecha del mismo por la red neuronal LSTM, desde 2017 hasta 2024, con fuga de información.

Con normalización Max- Min y sin fuga de información

Se obtuvo una buena predicción de la evolución del histórico del índice de Hurst por parte de la red LSTM (véase figura 6) respecto el histórico del índice de Hurst obtenido teóricamente del S&P500 desde aproximadamente 2017 hasta el 2024 (que es el lapso de tiempo del 20% de los datos para la comparación) tan buena como la realizada con fuga de información obteniendo estos parámetros de comparación: MAE : 0.0109, MSE : 0.0002, $RMSE$: 0.0136, R^2 : 0.9021,

$MAPE$: 2.03% este comportamiento tan bueno en la predicción a pesar no tener la ayuda de la fuga de información se debe a que la corrección en la normalización no afectó significativamente los resultados es que, en este caso, la fuga de datos probablemente no estaba introduciendo una ventaja significativa. Aquí hay varias explicaciones: Los datos de entrenamiento y prueba tienen distribuciones similares como estás trabajando con ventanas del índice de Hurst, los datos en el conjunto de prueba provienen del mismo proceso subyacente que los datos de entrenamiento. Si los datos de entrenamiento y prueba siguen distribuciones similares, entonces el impacto de la fuga de datos por normalización es mínimo. En otras palabras, aunque antes se usaba información del conjunto completo para escalar, la diferencia en la transformación fue pequeña.

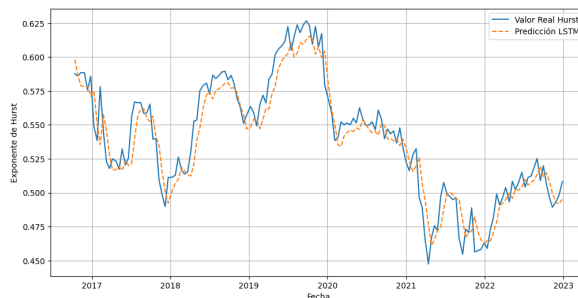


Figure 6: Gráfica de comparación entre el histórico del índice de Hurst teórico del S&P500 y la predicción hecha del mismo por la red neuronal LSTM, desde 2017 hasta 2024, sin fuga de información.

8.4 Desarrollo del cuarto objetivo específico

Después de aplicar lo dicho en la metodología del cuarto objetivo específico, se obtuvieron resultados muy satisfactorios como se puede ver en las figuras 7 y 8, la cual indica:

Con Normalización directa y fuga de información

Se obtuvo un comportamiento muy similar en la evolución histórica de la entropía de Shannon (véase figura 7), obtenido por la red LSTM, en comparación con la serie histórica obtenida teóricamente de la entropía de Shannon del S&P500, para el tiempo entre 2017 y 2024, que es el 20% de los datos para comparación, siendo estos sus parámetros de comparación: MAE : 0.0114, MSE : 0.0016, $RMSE$: 0.0397, R^2 : 0.9916, $MAPE$: 0.31%, mostrando así que la predicción hecha por la red LSTM es muy buena y veremos que comparada con el método de normalización sin fuga de datos que es muy similar, cuya explicación está después del siguiente análisis.

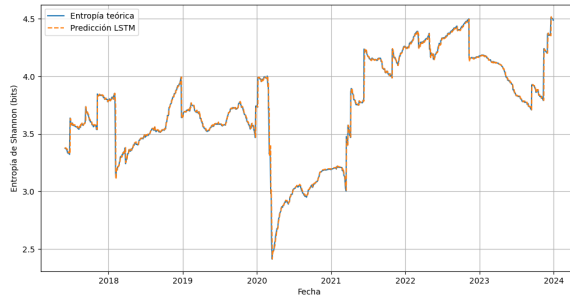


Figure 7: Gráfica de comparación entre el histórico de la entropía de Shannon teórica del S&P500 y la predicción hecha de la misma por la red neuronal LSTM, desde 2017 hasta 2024, con fuga de información.

Con normalización Max- Min y sin fuga de información

Se obtuvo una buena predicción de la evolución del histórico de la entropía de Shannon por parte de la red LSTM (véase figura 8) respecto al histórico de la entropía de Shannon obtenida teóricamente del S&P500 desde aproximadamente 2017 hasta el 2024 (que es el lapso de tiempo del 20% de los datos para la comparación) tan buena como la realizada con fuga

de información obteniendo estos parámetros de comparación: MAE : 0.0237, MSE : 0.0021, $RMSE$: 0.0456, R^2 : 0.9891, $MAPE$: 0.68% este comportamiento tan bueno en la predicción a pesar no tener la ayuda de la fuga de información se debe a que la corrección en la normalización no afectó significativamente los resultados es que, en este caso, la fuga de datos probablemente no estaba introduciendo una ventaja significativa. Aquí hay varias explicaciones: Los datos de entrenamiento y prueba tienen distribuciones similares como estás trabajando con ventanas del índice de Hurst, los datos en el conjunto de prueba provienen del mismo proceso subyacente que los datos de entrenamiento. Si los datos de entrenamiento y prueba siguen distribuciones similares, entonces el impacto de la fuga de datos por normalización es mínimo. En otras palabras, aunque antes se usaba información del conjunto completo para escalar, la diferencia en la transformación fue pequeña.

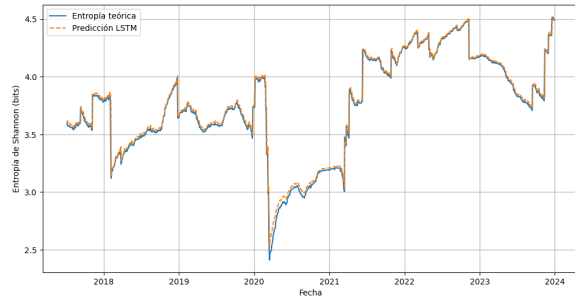


Figure 8: Gráfica de comparación entre el histórico de la entropía de Shannon teórica del S&P500 y la predicción hecha de la misma por la red neuronal LSTM, desde 2017 hasta 2024, sin fuga de información.

8.5 Desarrollo del quinto objetivo específico

Después de aplicar lo dicho en la metodología del quinto objetivo específico, se obtuvieron resultados muy satisfactorios como se puede ver en las figuras 9 y 10, la cual indica:

Con Normalización directa y fuga de información

Se obtuvo un comportamiento muy similar en la evolución histórica del parámetro de fractalidad (véase figura 9), obtenido por la red LSTM, en comparación con la serie del histórica obtenida teóricamente del parámetro de fractalidad del S&P500, para el tiempo entre 2017 y 2024, que es el 20% de los datos para comparación, siendo estos sus parámetros de comparación: MAE : 0.0021, MSE : 0.0000, $RMSE$: 0.0028, R^2 : 0.9841, $MAPE$: 0.20%, mostrando así que la predicción hecha por la red LSTM es muy buena y veremos que comparada con el método de normalización sin fuga de datos que es muy similar, cuya explicación esta después del siguiente análisis .

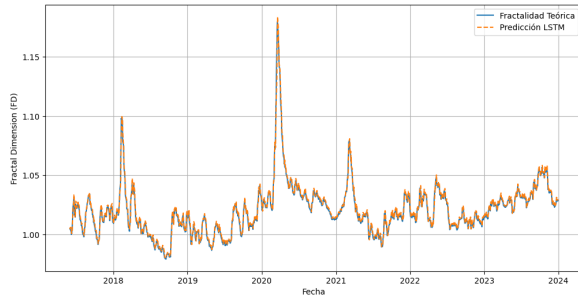


Figure 9: Gráfica de comparación entre el histórico del parámetro de fractalidad teórico del S&P500 y la predicción hecha del mismo por la red neuronal LSTM, desde 2017 hasta 2024, con fuga de información.

Con normalización Max- Min y sin fuga de información

Se obtuvo una buena predicción de la evolución del histórico del parámetro de fractalidad por parte de la red LSTM (véase figura 10) respecto el histórico del parámetro de fractalidad obtenida teóricamente del S&P500 desde aproximadamente 2017 hasta el 2024 (que es el lapso de tiempo del 20% de los datos para la comparación) tan buena como la realizada con fuga

de información obteniendo estos parámetros de comparación: MAE : 0.0018, MSE : 0.0000, $RMSE$: 0.0025, R^2 : 0.9872, $MAPE$: 0.18% este comportamiento tan bueno en la predicción a pesar no tener la ayuda de la fuga de información se debe a que la corrección en la normalización no afectó significativamente los resultados es que, en este caso, la fuga de datos probablemente no estaba introduciendo una ventaja significativa. Aquí hay varias explicaciones: Si la LSTM es capaz de generalizar bien, pequeñas diferencias en la normalización no deberían afectar mucho el desempeño. Dado que la fractalidad es una métrica calculada a partir de ventanas temporales, su variación podría ser suave y permitir que el modelo prediga correctamente incluso con una normalización diferente. Cuando hay fuga de información, suele pasar que los valores de normalización (mínimos y máximos) están influidos por datos futuros. Sin embargo, parece que la normalización sin fuga sigue siendo lo suficientemente representativa, lo que indica que: La fractalidad no tiene valores extremos que sesguen la normalización. El rango de variabilidad de los datos es relativamente estable, por lo que la normalización hecha solo con el conjunto de entrenamiento no cambia drásticamente.

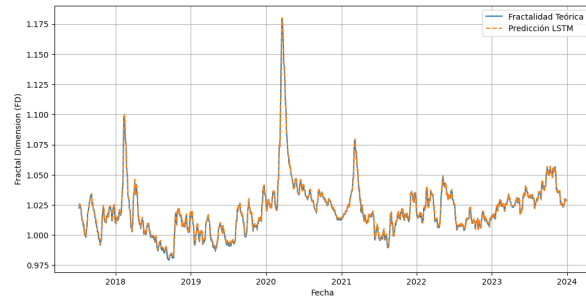


Figure 10: Gráfica de comparación entre el histórico del parámetro de fractalidad teórico del S&P500 y la predicción hecha del mismo por la red neuronal LSTM, desde 2017 hasta 2024, sin fuga de información.

8.6 Desarrollo del sexto objetivo específico

Después de aplicar lo dicho en la metodología del sexto objetivo específico, se obtuvieron resultados muy satisfactorios como se puede ver en las figuras 11 y 12, la cual indica:

Con Normalización directa y fuga de información

Se obtuvo un comportamiento muy similar en la evolución histórica del parámetro de volatilidad (véase figura 11), obtenido por la red LSTM, en comparación con la serie del histórico obtenida teóricamente del parámetro de volatilidad del S&P500, para el tiempo entre 2017 y 2024, que es el 20% de los datos para comparación, siendo estos sus parámetros de comparación: MAE : 0.0054, MSE : 0.0001, $RMSE$: 0.0079, R^2 : 0.9942, $MAPE$: 2.86%, mostrando así que la predicción hecha por la red LSTM es muy buena y veremos que comparada con el método de normalización sin fuga de datos que es muy similar, cuya explicación esta después del siguiente análisis .

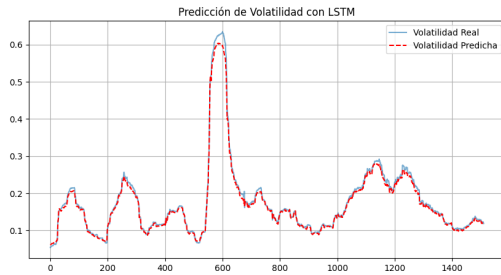


Figure 11: Gráfica de comparación entre el histórico del parámetro de volatilidad teórico del S&P500 y la predicción hecha del mismo por la red neuronal LSTM, desde 2017 hasta 2024, con fuga de información.

Con normalización Max- Min y sin fuga de información

Se obtuvo una buena predicción de la evolución del histórico del parámetro de volatilidad por parte de la red LSTM (véase figura 12) respecto el histórico del parámetro de volatilidad obtenido teóricamente del S&P500 desde aproximadamente 2017 hasta el 2024 (que es el lapso de tiempo del 20% de los datos para la comparación) tan buena como la realizada con fuga de información obteniendo estos parámetros de comparación: MAE : 0.0022, MSE : 0.0000, $RMSE$: 0.0039, R^2 : 0.9986, $MAPE$: 1.17% este comportamiento tan bueno en la predicción a pesar no tener la ayuda de la fuga de información se debe a que la corrección en la normalización no afectó significativamente los resultados es que, en este caso, la fuga de datos probablemente no estaba introduciendo una ventaja significativa. Aquí hay varias explicaciones: Si la LSTM es capaz de generalizar bien, pequeñas diferencias en la normalización no deberían afectar mucho el desempeño. Dado que la volatilidad es una métrica calculada a partir de ventanas temporales, su variación podría ser suave y permitir que el modelo prediga correctamente incluso con una normalización diferente. Cuando hay fuga de información, suele pasar que los valores de normalización (mínimos y máximos) están influidos por datos futuros. Sin embargo, parece que la normalización sin fuga sigue siendo lo suficientemente representativa, lo que indica que: La volatilidad no tiene valores extremos que sesguen la normalización. El rango de variabilidad de los datos es relativamente estable, por lo que la normalización hecha solo con el conjunto de entrenamiento no cambia drásticamente.



Figure 12: Gráfica de comparación entre el histórico del parámetro de volatilidad teórico del S&P500 y la predicción hecha del mismo por la red neuronal LSTM, desde 2017 hasta 2024, sin fuga de información.

9 Análisis de Resultados

En general, la LSTM generaliza bien y No Está Memorizando Si el modelo dependiera en os diferentes casos fuertemente de la fuga de datos, la predicción sin fuga se deterioraría. Sin embargo, el hecho de que la precisión siga siendo casi la misma para los diferentes casos indica que la LSTM está aprendiendo patrones reales en la serie temporal, y no simplemente ajustándose a valores específicos de la normalización.

9.1 Análisis de Resultados del primer objetivo específico analítico

Después del extenso análisis y desarrollo analítico realizados tanto en el apartado teórico como en el desarrollo del primer objetivo, se obtuvo como resultado la demostración derivada del calculo tensorial para evitar el problema del desvanecimiento del gradiente en el flujo de información de cada una de las celdas que componen la red neuronal de tipo LSTM, dando esto como resultado una teoría solida y robusta que se implementa en una red neuronal para que esta pueda tener una buena memoria a largo plazo de una manera eficiente, y ademas se hizo el desarrollo del surgimiento desde el calculo tensorial de las puertas lógicas (puerta de entrada, de salida

y olvido) mostrando así lo interrelacionado que esta el calculo tensorial con las redes neuronales de tipo LSTM:

9.2 Análisis de Resultados del segundo objetivo específico

Como se pudo ver en desarrollo del segundo objetivo, se logro cumplir lo planteado y de una, manera muy buena (tanto si se normalizo para que hubiera o no hubiera fuga de información, aunque su diferencia se nota, su predicción en los dos casos es buena) lo que nos indica que el histórico de precios del S&P500 es una serie increíblemente bien portada, en lo que a mercado eficiente se refiere, esto gracias a que es el compendio de las 500 empresas con mejor comportamiento financiero en EE.UU (una de las economías mas fuertes y por ende mas eficientes del mundo), lo cual es muy útil para probar nuevas herramientas de análisis econofisico, razón por la cual es importante resaltar que si aplicamos la misma red neuronal (de una sola capa) a otra serie de precios mucho mas inestable e impredecible en el tiempo, no se logran los mismos resultados, lo cual no significa que el método no funcione si no que se debe adecuar mucho mejor la red neuronal para que logre predecir, pero esto por supuesto lleva mucha mas demanda de computo.

9.3 Análisis de Resultados del tercer objetivo específico

Ahora, analizando las gráficas correspondientes con los parametros comparativos de cada una, se puede notar la poca importancia que presenta el que halla fuga de datos o no a causas de la normalización, además de ver la calidad de las predicciones del comportamiento historico del indice de Hurst obtenido por la red neuronal, también podemos hacer el **análisis econofisico** del cambio en el tiempo del indice de Hurst, lo cual nos indica por ejemplo la crisis económica del 2021 debido a la pandemia por el Covid 19, mostrando la caída pronunciada desde finales de

2019 hasta inicios de 2021, pero en general, entre el 2017 y 2024 muestra una tendencia centrada al rededor del 0.53, lo cual nos indica que el S&P500 esta en una economía que tiende a ser una economía eficiente y por eso pudo recuperarse de la crisis causada por la pandemia.

9.4 Análisis de Resultados del cuarto objetivo específico

Ahora, analizando las gráficas correspondientes con los parámetros comparativos de cada una, se puede notar la poca importancia que presenta el que halla fuga de datos o no a causas de la normalización, además de ver la calidad de las predicciones del comportamiento histórico de la entropía de Shannon obtenida por la red neuronal, también podemos hacer el **análisis econofísico** del cambio en el tiempo de la entropía de Shannon, lo cual nos indica por ejemplo la crisis económica del 2021 debido a la pandemia por el Covid 19, mostrando como la entropía baja bruscamente dejándonos saber que en esa situación de crisis, los estados posibles del sistema se redujeron bastante, tendiendo a los estados de precios mas bajos, pero en general, entre el 2017 y 2024 muestra una tendencia un poco hacia el crecimiento indicando esto lo fuerte y ligado a la economía mundial que esta el indice del S&P500, lo cual nos indica que el S&P500 esta en una economía que tiende a ser una economía eficiente.

9.5 Análisis de Resultados del quinto objetivo específico

Ahora, analizando las gráficas correspondientes con los parámetros comparativos de cada una, se puede notar la poca importancia que presenta el que halla fuga de datos o no a causas de la normalización, además de ver la calidad de las predicciones del comportamiento histórico del parámetro de fractalidad obtenido por la red neuronal, también podemos hacer el **análisis econofísico** del cambio en el tiempo de la fractalidad, lo cual nos indica por ejemplo la cri-

sis económica del 2021 debido a la pandemia por el Covid 19, mostrando como la fractalidad crece bruscamente dejándonos saber que en esa situación de crisis, la dimensión fractal crece bastante, mostrando así que el mercado esta lejos de ser eficiente, pero en general, entre el 2017 y 2024 muestra una tendencia un poco hacia el decrecimiento indicando esto lo fuerte y ligado a la economía mundial que esta el indice del S&P500, lo cual nos indica que el S&P500 esta en una economía que tiende a ser una economía eficiente.

9.6 Análisis de Resultados del sexto objetivo específico

Ahora, analizando las gráficas correspondientes con los parámetros comparativos de cada una, se puede notar la poca importancia que presenta el que halla fuga de datos o no a causas de la normalización, además de ver la calidad de las predicciones del comportamiento histórico del parámetro de Volatilidad obtenido por la red neuronal, también podemos hacer el **análisis econofísico** del cambio en el tiempo de la volatilidad, lo cual nos indica por ejemplo la crisis económica del 2021 (alrededor de 450 al 510) debido a la pandemia por el Covid 19, mostrando como la volatilidad crece bruscamente dejándonos saber que en esa situación de crisis, los estados posibles del sistema se redujeron bastante, haciendo al sistema bastante predecible y fuera de lo que sería un mercado eficiente, tendiendo la volatilidad a aumentar, pero en general, entre el 2017 y 2024 muestra una tendencia hacia el decrecimiento indicando esto lo fuerte y ligado a la economía mundial que esta el indice del S&P500, llegando la volatilidad a un comportamiento prepandemia, lo cual nos indica que el S&P500 esta en una economía que tiende a ser una economía eficiente.

10 Conclusiones

10.1 Conclusión del primer objetivo específico

A partir del cálculo tensorial, es posible comprender cómo se construyen las puertas lógicas que conforman las celdas de una red LSTM, ya que estas redes neuronales recurrentes dependen de operaciones matriciales y tensores para gestionar la memoria y la propagación de la información en secuencias temporales. En esencia, las celdas LSTM están diseñadas para mitigar el problema del desvanecimiento del gradiente mediante un conjunto de puertas que regulan la actualización y el mantenimiento del estado de la memoria. Estas puertas—de entrada, olvido y salida—se definen matemáticamente a través de productos tensoriales entre los pesos de la red y las entradas secuenciales, junto con funciones de activación como la sigmoide y la tangente hiperbólica. La formulación en términos de tensores permite describir cómo la información fluye de manera diferenciable dentro de la red, asegurando que cada estado oculto capture dependencias de largo plazo en la secuencia. De manera más profunda, el cálculo tensorial proporciona una notación compacta y eficiente para modelar la interdependencia entre las distintas variables en una celda LSTM, facilitando su implementación en frameworks de deep learning como TensorFlow o PyTorch. En última instancia, la relación entre los tensores y las puertas lógicas de la LSTM demuestra que estas redes no son simplemente heurísticas empíricas para modelar series temporales, sino que tienen una base matemática rigurosa que explica su capacidad para aprender patrones complejos en datos secuenciales.

10.2 Conclusión del segundo objetivo específico

De los resultados obtenidos para el segundo objetivo específico, se puede concluir que las herramientas de computo proporcionadas por el

entorno de Google colab, son mas que suficientes para ejecutar de manera rápida y simple el código necesario para que una red neuronal de tipo LSTM prediga el comportamiento del histórico de precios del S&P500, ya sea usando normalización normal con consecuencia de fuga de datos, o con normalización tipo Max-Min para que no halla fuga de datos, mostrando así que el equipo de computo con hardware mas avanzado no es necesario y que con solo un dispositivo con conexión a internet, se puede lograr lo obtenido para el segundo objetivo específico. Además, también se puede concluir el porque las compañías financieras gastan grandes recursos en software y hardware para la predicción mas exacta posible del futuro precio en activos financieros, a muy corto plazo (mili segundos) ya que por la consecuencia del profecía auto cumplida, no tiene sentido predecir demasiado adelante en el futuro.

10.3 Conclusión del tercer objetivo específico

De los resultados obtenidos para el segundo objetivo específico, se puede concluir que las herramientas de computo proporcionadas por el entorno de Google colab, son mas que suficientes para ejecutar de manera rápida y simple el código necesario para que una red neuronal de tipo LSTM prediga el comportamiento del histórico del indice de hurst del S&P500, ya sea usando normalización normal con consecuencia de fuga de datos, o con normalización tipo Max-Min para que no halla fuga de datos, mostrando así que el equipo de computo con hardware mas avanzado no es necesario y que con solo un dispositivo con conexión a internet, se puede lograr lo obtenido para el segundo objetivo específico. Además, también se puede concluir el porque las compañías financieras gastan grandes recursos en software y hardware para la predicción mas exacta posible del futuro precio en activos financieros, a muy corto plazo (mili segundos) ya que por la consecuencia del profecía auto cumplida, no tiene sentido predecir demasiado adelante

en el futuro..

10.4 Conclusión del cuarto objetivo específico

De los resultados obtenidos para el segundo objetivo específico, se puede concluir que las herramientas de computo proporcionadas por el entorno de Google colab, son mas que suficientes para ejecutar de manera relativamente rápida y simple el código necesario para que una red neuronal de tipo LSTM prediga el comportamiento del histórico de la entropía de Shannon del S&P500, ya sea usando normalización normal con consecuencia de fuga de datos, o con normalización tipo Max-Min para que no halla fuga de datos, mostrando así que el equipo de computo con hardware mas avanzado no es necesario y que con solo un dispositivo con conexión a internet, se puede lograr lo obtenido para el segundo objetivo específico. Ademas, también se puede concluir el porque las compañías financieras gastan grandes recursos en software y hardware para la predicción mas exacta posible del futuro precio en activos financieros, a muy corto plazo (mili segundos) ya que por la consecuencia del profecía auto cumplida, no tiene sentido predecir demasiado adelante en el futuro.

10.5 Conclusión del quinto objetivo específico

De los resultados obtenidos para el segundo objetivo específico, se puede concluir que las herramientas de computo proporcionadas por el entorno de Google colab, son mas que suficientes para ejecutar de manera rápida y simple el código necesario para que una red neuronal de tipo LSTM prediga el comportamiento del histórico de del parametro de fractalidad del S&P500, ya sea usando normalización normal con consecuencia de fuga de datos, o con normalización tipo Max-Min para que no halla fuga de datos, mostrando así que el equipo de computo con hardware mas avanzado no es necesario y que con solo un dispositivo con conexión a internet,

se puede lograr lo obtenido para el segundo objetivo específico. Ademas, también se puede concluir el porque las compañías financieras gastan grandes recursos en software y hardware para la predicción mas exacta posible del futuro precio en activos financieros, a muy corto plazo (mili segundos) ya que por la consecuencia del profecía auto cumplida, no tiene sentido predecir demasiado adelante en el futuro.

10.6 Conclusión del sexto objetivo específico

De los resultados obtenidos para el segundo objetivo específico, se puede concluir que las herramientas de computo proporcionadas por el entorno de Google colab, son mas que suficientes para ejecutar de manera rápida y simple el código necesario para que una red neuronal de tipo LSTM prediga el comportamiento del histórico del parametro de volatilidad del S&P500, ya sea usando normalización normal con consecuencia de fuga de datos, o con normalización tipo Max-Min para que no halla fuga de datos, mostrando así que el equipo de computo con hardware mas avanzado no es necesario y que con solo un dispositivo con conexión a internet, se puede lograr lo obtenido para el segundo objetivo específico. Ademas, también se puede concluir el porque las compañías financieras gastan grandes recursos en software y hardware para la predicción mas exacta posible del futuro precio en activos financieros, a muy corto plazo (mili segundos) ya que por la consecuencia del profecía auto cumplida, no tiene sentido predecir demasiado adelante en el futuro.

11 Anexos

11.1 ACLARACIÓN

Todos los códigos empleados fueron explicados en la parte de metodología y ademas también fueron citados debidamente a lo largo del documento en el apartado bibliográfico [8], ahí se podrá encontrar todos lo códigos no solamente

debidamente comentados si no explicados otra vez con ventanas de texto, REPITO, no hace falta mas anexos, lo códigos y sus debidas explicaciones ademas de los explicados en este documento, están en [8].

11.2 Anexo para el primer objetivo específico

No hay anexo ya que los cálculos se realizaron sobre la marcha usando las referencias mencionadas en el apartado teórico y en el desarrollo del primer objetivo específico analítico.

11.3 Anexo para el segundo objetivo específico

También se hicieron códigos para ver como la red neuronal con normalización con fuga de datos, de tres capas y una sola capa no son suficiente para predecir 5 días en el futuro el histórico del precio del S&P500 (Ver celda 3 y 4 de [8]) obteniendo la gráfica de la figura 13.

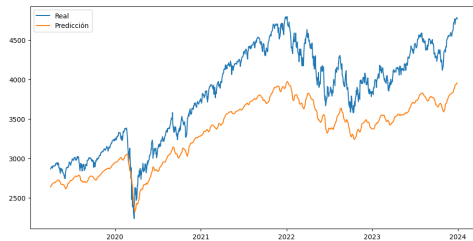


Figure 13: Comparación de serie real y predicha por la red LSTM con 3 capas, sobre el S&P500.

11.4 Anexo para el tercer objetivo específico

Se realizo un código para calcular teóricamente la serie de cambio en el tiempo del indice de Hurts sobre una distribución gaussiana no correlacionada con la misma cantidad de datos que tiene el histórico de precios del S&P500 para corroborar que el calculo teórico es el correcto y que

sea un buen alimento para la red neuronal, obteniendo la curva azul de la gráfica en la figura 4 (ver celda 13 de [9])

11.5 Anexo para el cuarto objetivo específico

Se realizo un código para calcular teóricamente la serie de cambio en el tiempo del indice de Hurts sobre una distribución gaussiana no correlacionada con la misma cantidad de datos que tiene el histórico de precios del S&P500 para corroborar que el calculo teórico es el correcto y que sea un buen alimento para la red neuronal, obteniendo la curva azul de la gráfica en la figura 4 (ver celda 14 de [9])

12 Bibliografía

References

- [1] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, pp. 1735-1780, 1997.
- [2] Viviana María Oquendo Patiño: "Redes Neuronales Artificiales en las Ciencias Económicas" diciembre de 2011
- [3] Shannon, C. E. (1948). *A Mathematical Theory of Communication*. Bell System Technical Journal.
- [4] Mantegna, R. N., & Stanley, H. E. (1999). *Introduction to Econophysics*. Cambridge University Press.
- [5] Mandelbrot, B. B. (1967). *The variation of certain speculative prices*. The Journal of Business.
- [6] Peters, E. E. (1994). *Fractal market analysis*. John Wiley & Sons.
- [7] E. Fama, "Efficient Capital Markets: A Review of Theory and Empirical Work," *Journal of Finance*, vol. 25, pp. 383-417, 1970.

- [8] Colab Notebook: Implementación de los Modelos,
A. Robinson, 2025.
- [9] Yahoo Finance S&P500
- [10] Video explicativo de la red neuronal tipo LSTM
- [11] Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press.
- [12] C. C. Aggarwal, *Neural Networks and Deep Learning: A Textbook*. Springer, 2018.